GEORG-AUGUST-UNIVERSITÄT GÖTTINGEN IN PUBLICA COMMODA SEIT 1737

COMPUTATIONAL CELL ANALYTICS

Deep Learning for Computer Vision

Lecture 7: Visualizing NNs & Style Transfer

Constantin Pape

Institute of Computer Science, Georg August Universität Göttingen

@cppape.bsky.social

https://user.informatik.uni-goettingen.de/~pape41/

Organization: Exercise Schedule

- Exercise 3: Transfer learning for medical imaging
  - Was due just now.
  - We will present results from test-set submission in the lecture next week.

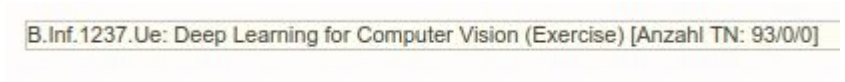
- Exercise 4: U-Net for segmentation and denoising
  - Available on Stud.IP
  - **Due date: Jan 6th (Next year!)**

TODO: final reminder FlexNow

Final reminder: Sign up to FlexNow!

Deadline for signing up in FlexNow is 12.12.

Sign up now to get credits!



(Not relevant for PhD students)

Agenda for today

- Visualizing neural networks
  - Feature inversion
  - Activity maximization
  - Attribution

- Texture synthesis, style transfer and perceptual loss functions

Visualizing neural networks

What do the hidden layers do?

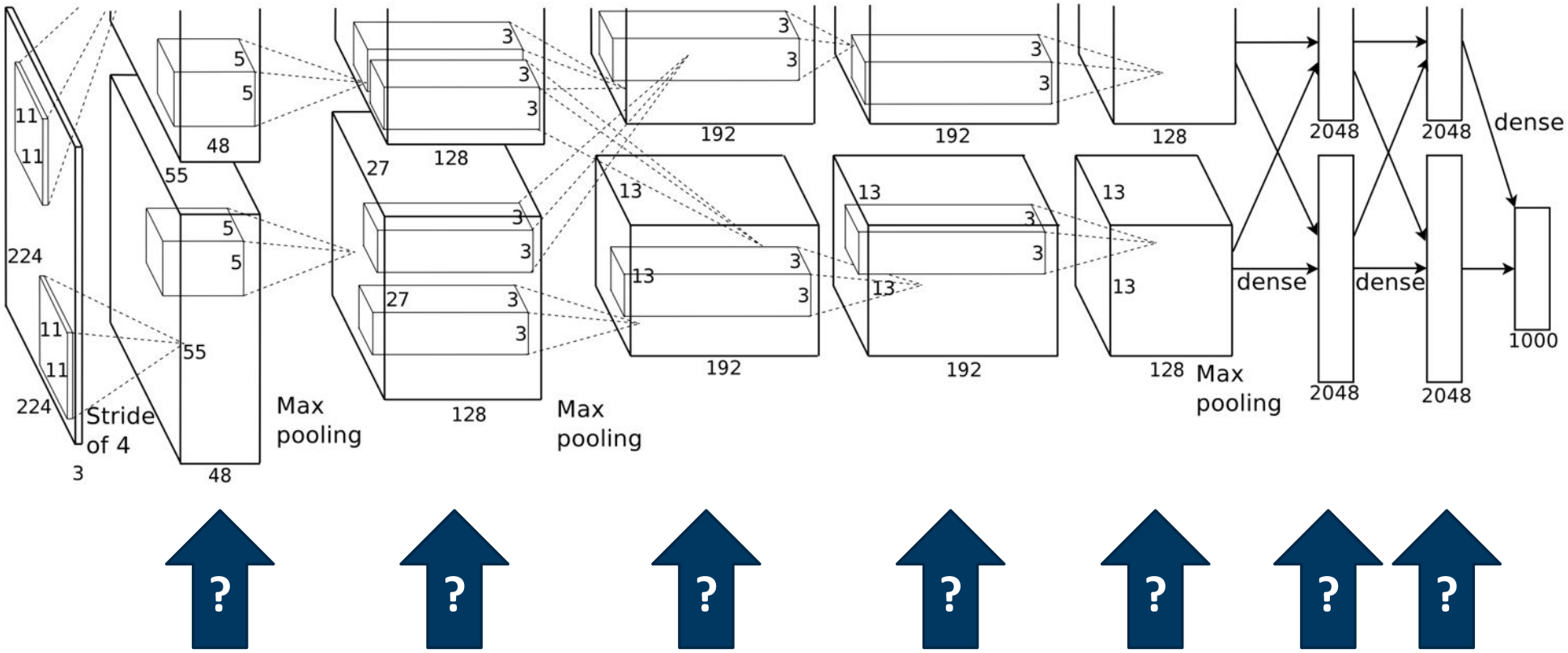


Image source: Krizhekvsky et al., NeurIPS 2012: “ImageNet Classification with Deep Convolutional Neural Networks”

Visualize filters: easy for first layer

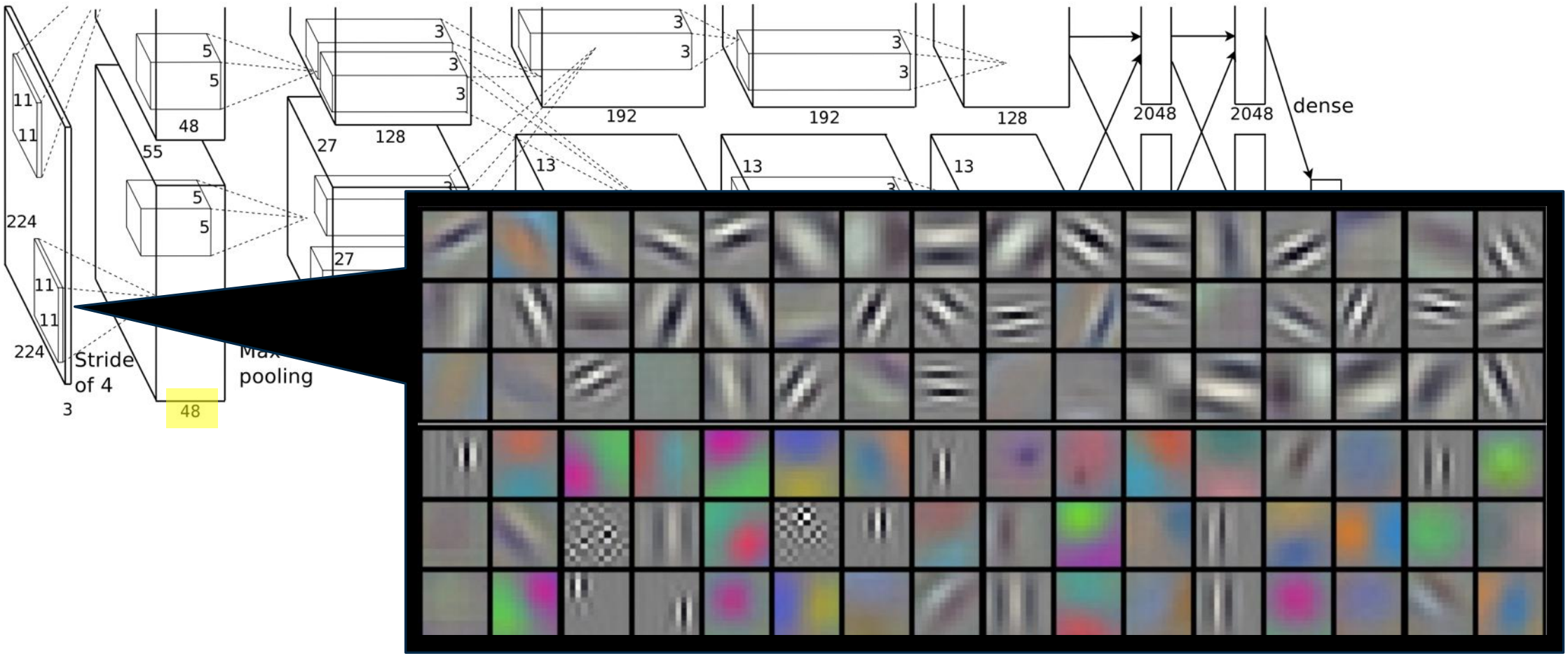
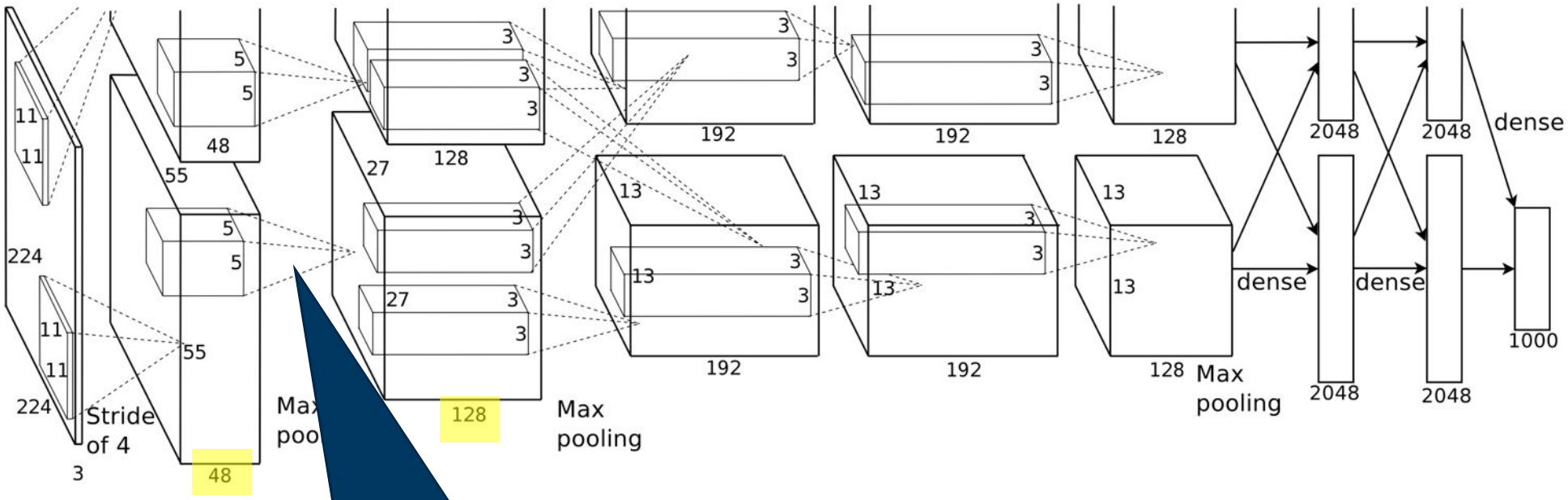


Image source: Krizhekvsky et al., NeurIPS 2012: “ImageNet Classification with Deep Convolutional Neural Networks”

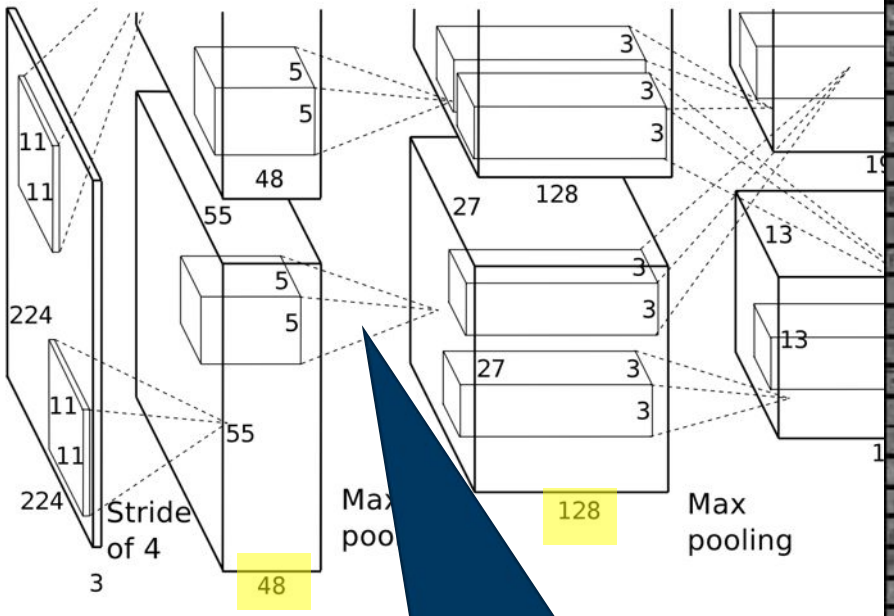
What about the second layer?



Contains 48 × 128 filters
→ Visualization not very useful

Image source: Krizhekvsky et al., NeurIPS 2012: “ImageNet Classification with Deep Convolutional Neural Networks”

What about the seco



Contains 48 × 128 filters
→ Visualization not very useful

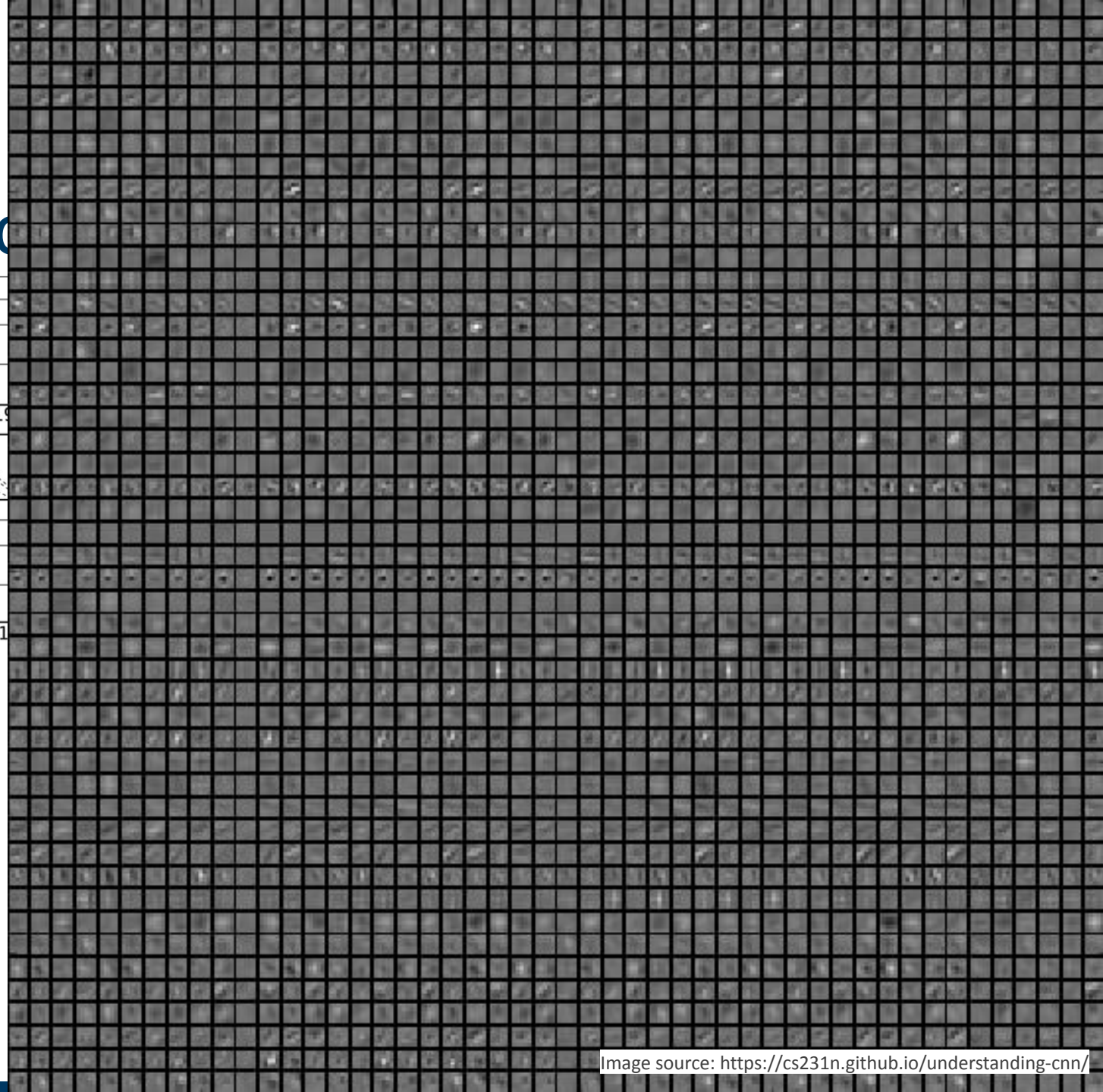


Image source: https://cs231n.github.io/understanding-cnn/

Visualize activations?

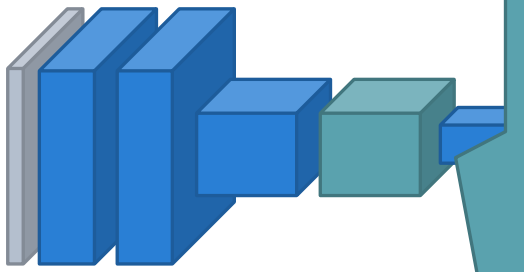
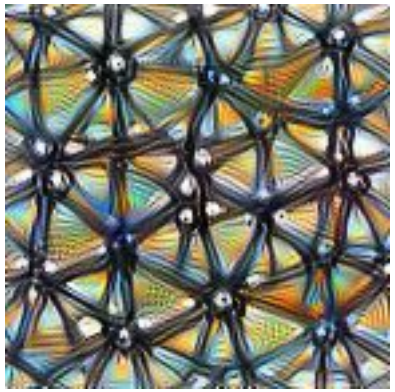


Image source: https://machinelearningmastery.com/how-to-visualize-filters-and-feature-maps-in-convolutional-neural-networks/

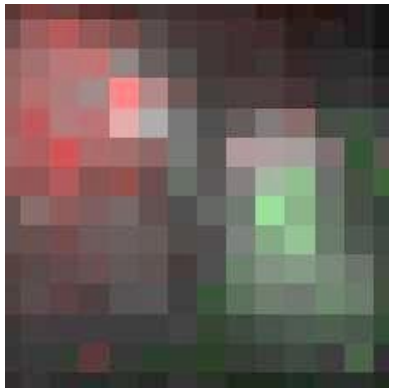
How do CNNs form decisions?



What features do individual units “look for”?

Feature visualization

“cat”



What image features are responsible for the network’s decision?

Attribution

Image source: Olah et al. (2017) https://distill.pub/2017/feature-visualization/

Why feature visualization?

Dataset Examples show us what neurons respond to in practice

Optimization isolates the causes of behavior from mere correlations. A neuron may not be detecting what you initially thought.

Baseball—or stripes?
mixed4a, Unit 6

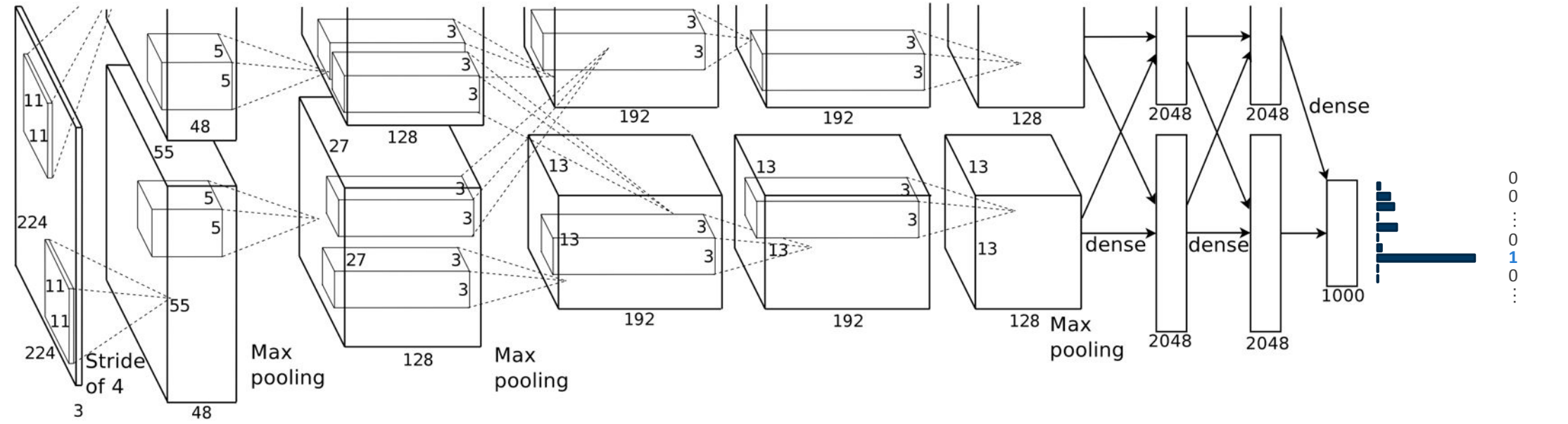
Animal faces—or snouts?
mixed4a, Unit 240

Clouds—or fluffiness?
mixed4a, Unit 453

Buildings—or sky?
mixed4a, Unit 492

Olah et al. (2017) https://distill.pub/2017/feature-visualization/

Information loss from image to class labels: How about intermediate layers?

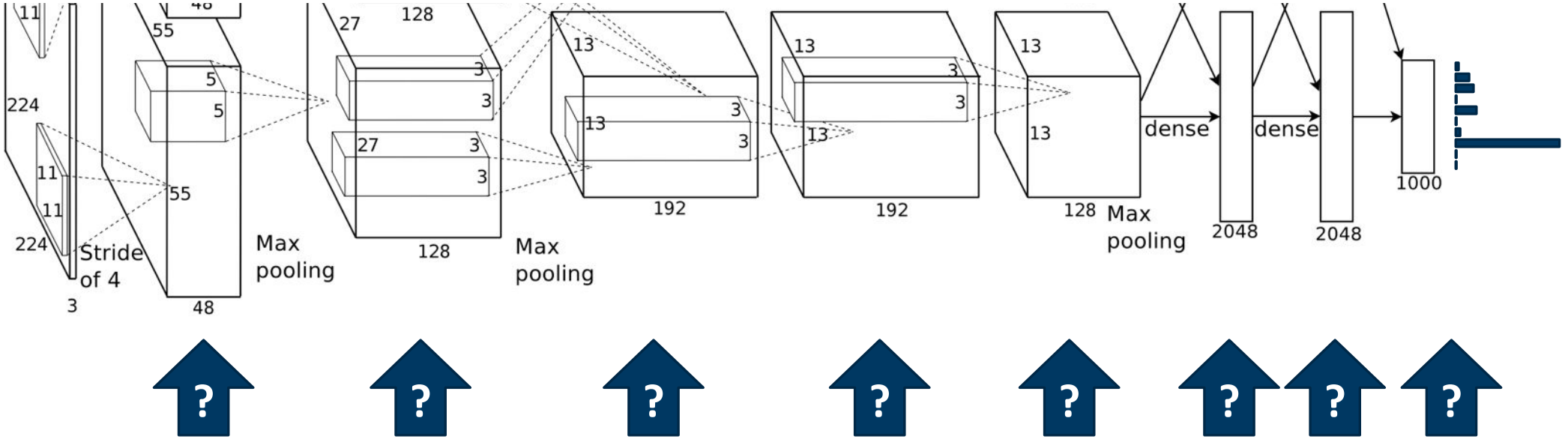


$224 \times 224 \times 24$
$= 1{,}204{,}224$ bits
of information

10 bits

Image source: Krizhekvsky et al., NeurIPS 2012: “ImageNet Classification with Deep Convolutional Neural Networks”

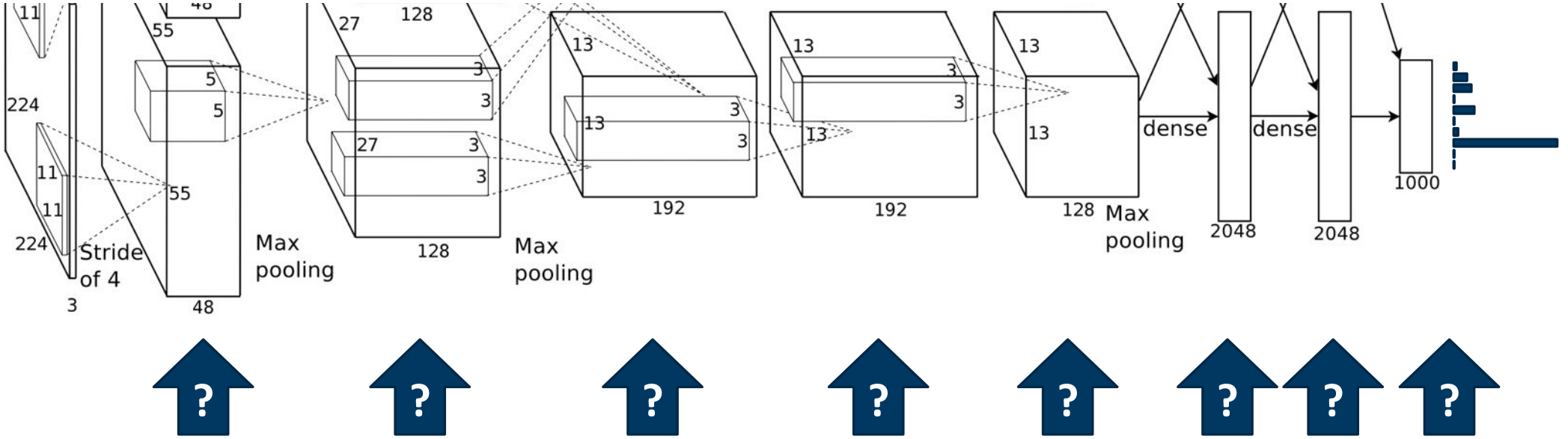
How do we visualize features?



Goal: visualize feature representation of any layer/unit!

Image source: Krizhekvsky et al., NeurIPS 2012: “ImageNet Classification with Deep Convolutional Neural Networks”

How do we visualize features?



Goal: visualize feature representation of any layer/unit!

- Inverting representation: reconstruct input by minimizing difference of feature responses
- Activation maximization: find input that maximizes activation

Image source: Krizhekvsky et al., NeurIPS 2012: “ImageNet Classification with Deep Convolutional Neural Networks”

Inverting CNN representations

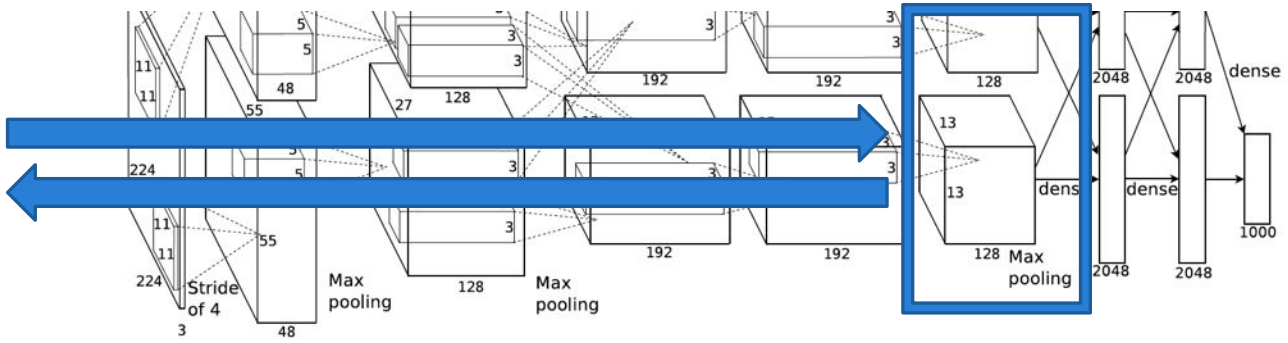
Image: $\mathbf{x}$

?

$\Phi^{-1}(\Phi(\mathbf{x}))$

In general not unique!

$\Phi(\mathbf{x})$

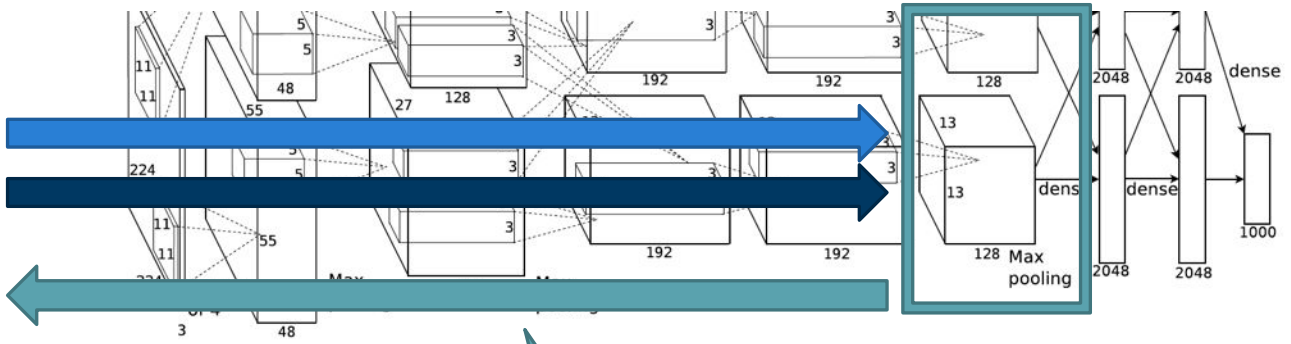


Mahendran & Vedaldi: “Understanding Deep Image Representations by Inverting Them.” CVPR 2015

Inverting CNN representations

Image: $\mathbf{x}$

$$\Phi(\mathbf{x}) = \Phi(\mathbf{x}^*)$$



$$L(\mathbf{x}, \mathbf{x}^*) = [\Phi(\mathbf{x}) - \Phi(\mathbf{x}^*)]^2$$

$$\hat{\mathbf{x}} = \arg\min L(\mathbf{x}, \mathbf{x}^*) + \lambda \mathcal{R}(\mathbf{x}^*)$$

Found by gradient descent

Backpropagate gradient

Iterate

1. Forward pass image
2. Backprop gradient
3. Update image

Mahendran & Vedaldi: "Understanding Deep Image Representations by Inverting Them." CVPR 2015

Inverting CNN representations

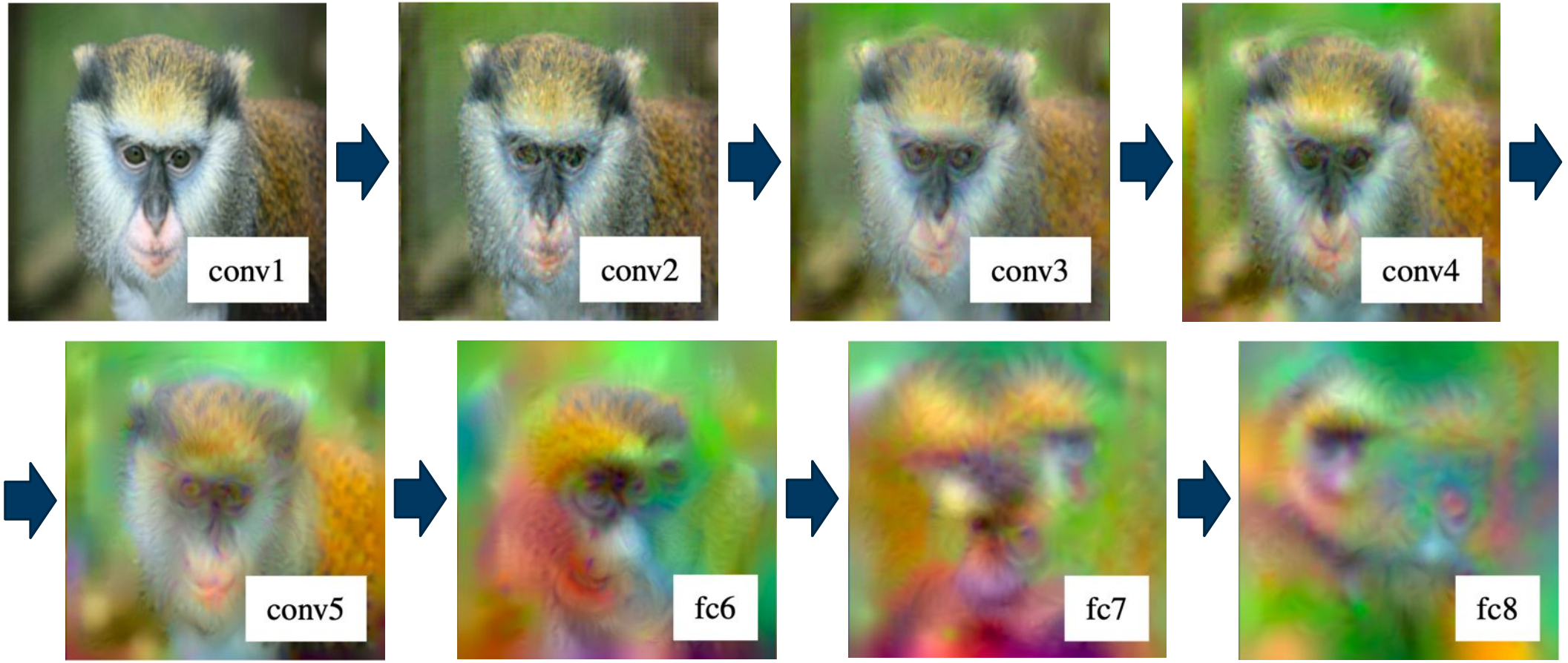
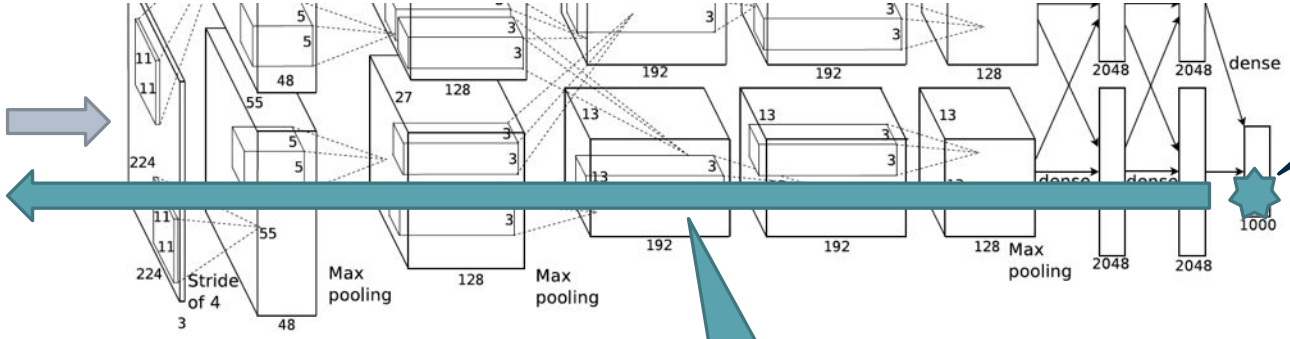


Image source: Mahendran & Vedaldi: “Understanding Deep Image Representations by Inverting Them.” CVPR 2015

Visualizing CNN features by activity maximization

Image $\mathbf{x}$

?



Goal: Maximize activation

Bell pepper

$$\mathbf{x}^* = \arg\max_{\mathbf{x}} \Phi(\mathbf{x}) - \mathcal{R}(\mathbf{x})$$

Found by gradient ascent

Backpropagate gradient

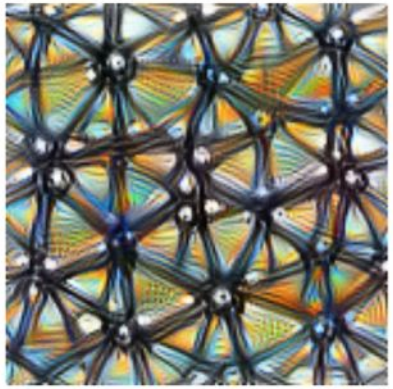
Iterate

1. Forward pass image
2. Backprop gradient
3. Update image

Activity maximization

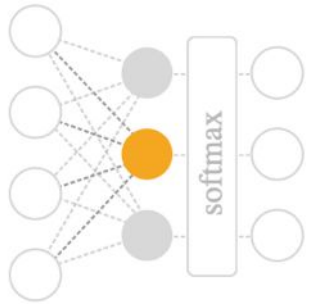
n layer index
x,y spatial position
z channel index
k class index

Neuron
`layer`$_n$`[x,y,z]`

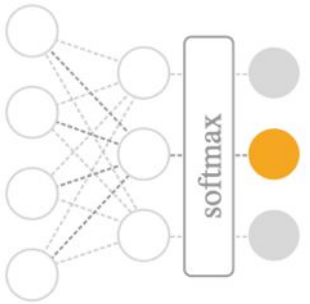


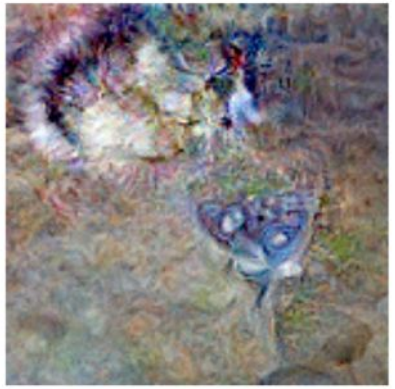
Channel
`layer`$_n$`[:,:,z]`

Layer/DeepDream
`layer`$_n$`[:,:,:]`^{2}



Class Logits
`pre_softmax[k]`



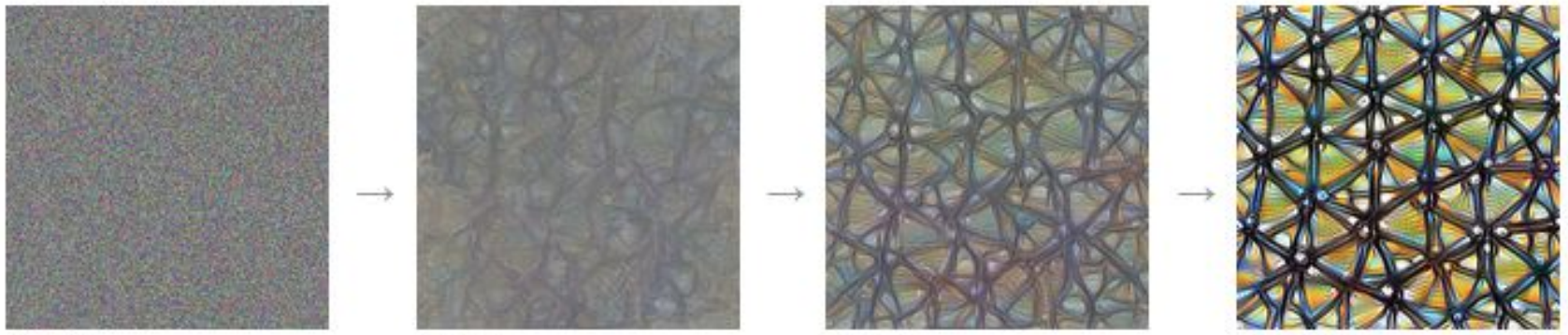


Class Probability
`softmax[k]`

Olah et al. (2017) https://distill.pub/2017/feature-visualization/

Activity maximization

Starting from random noise, we optimize an image to activate a particular neuron (layer mixed4a, unit 11).



Olah et al. (2017) https://distill.pub/2017/feature-visualization/

Naïve activity maximization doesn't work well

Objective being optimized: $\mathbf{x}^* = \arg\max_{\mathbf{x}} \Phi(\mathbf{x})$

Optimization produces artefacts / exaggerates high-frequency patterns

Image source: Olah et al. (2017) https://distill.pub/2017/feature-visualization/

Naïve activity maximization doesn't work well

Regularized objective being optimized: $\mathbf{x}^* = \arg\max_{\mathbf{x}} \Phi(\mathbf{x}) - \mathcal{R}(\mathbf{x})$

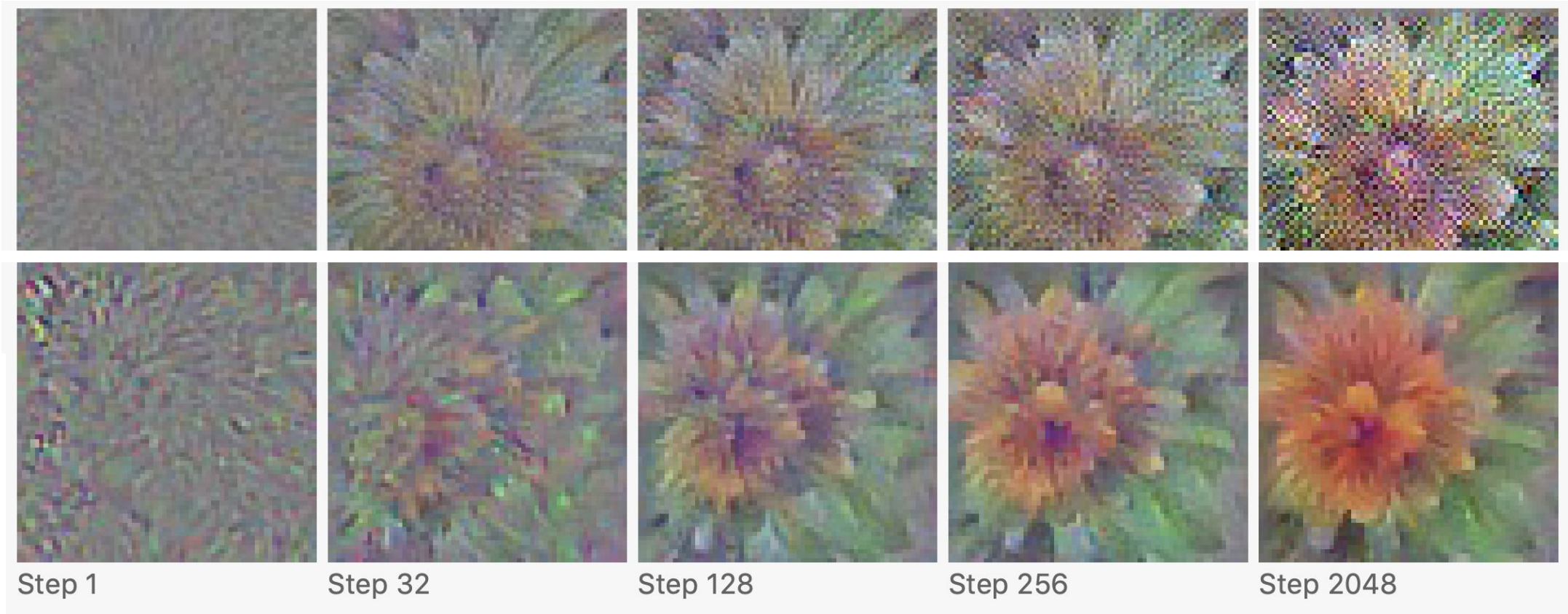
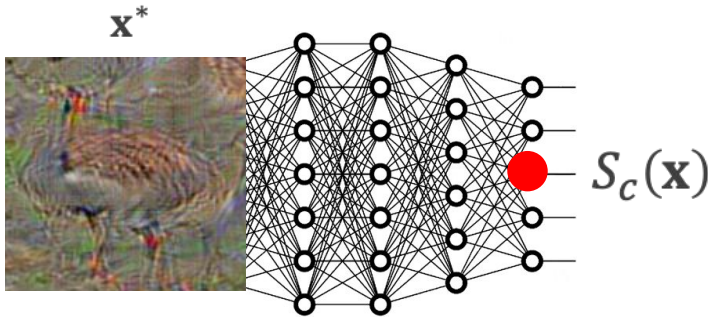


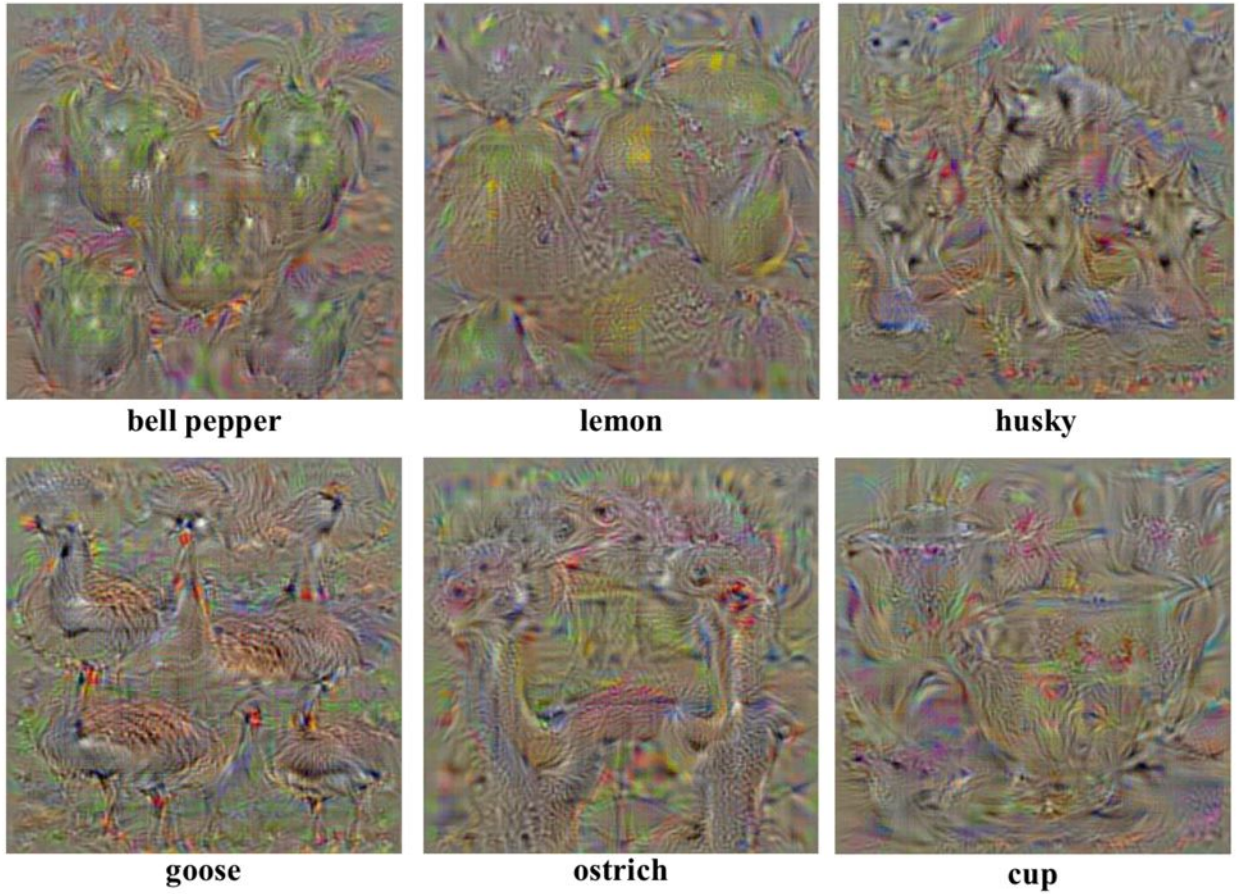
Image source: Olah et al. (2017) https://distill.pub/2017/feature-visualization/

Visualizing classification layer

Using L2 regularization:

$$\mathbf{x}^* = \arg\max_{\mathbf{x}} S_c(\mathbf{x}) - \lambda \|\mathbf{x}\|_2^2$$





Simonyan, Vedaldi, Zisserman, *ICLR Workshops* 2014

Regularization: key to get good reconstructions

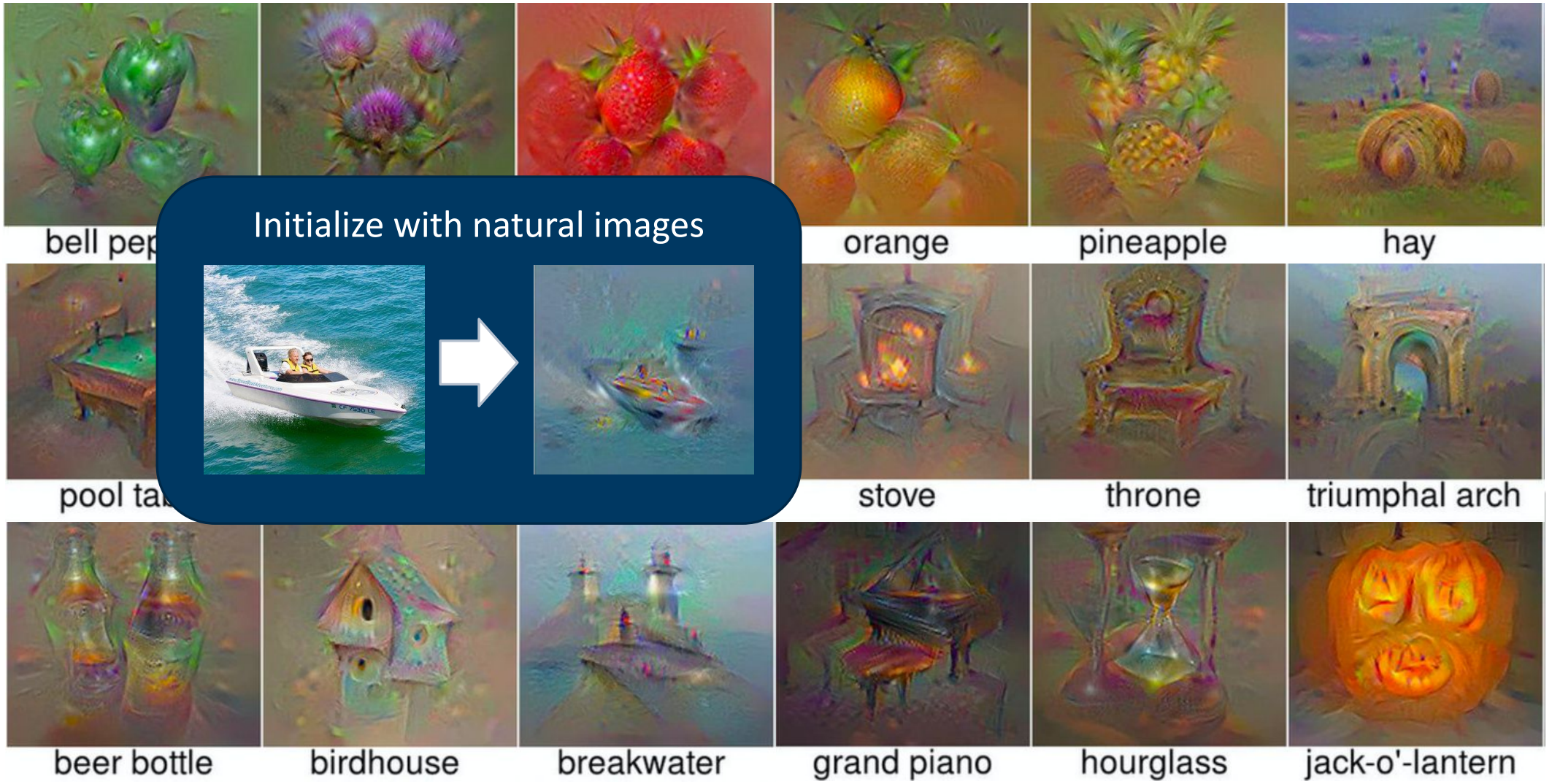


Image source: Nguyen et al., “Multifaceted Feature Visualization …”, https://arxiv.org/abs/1602.03616

Regularization: key to get good reconstructions

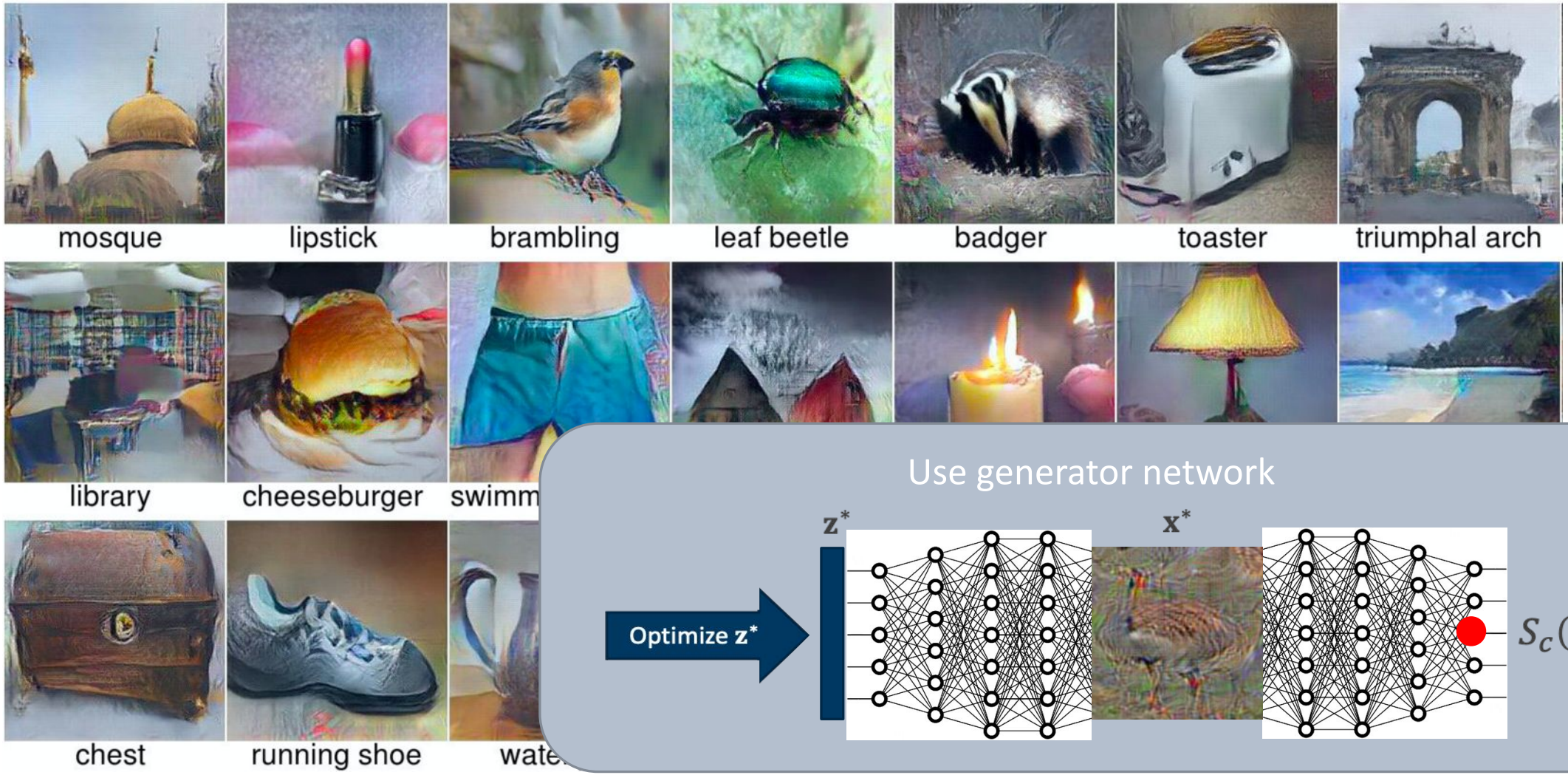
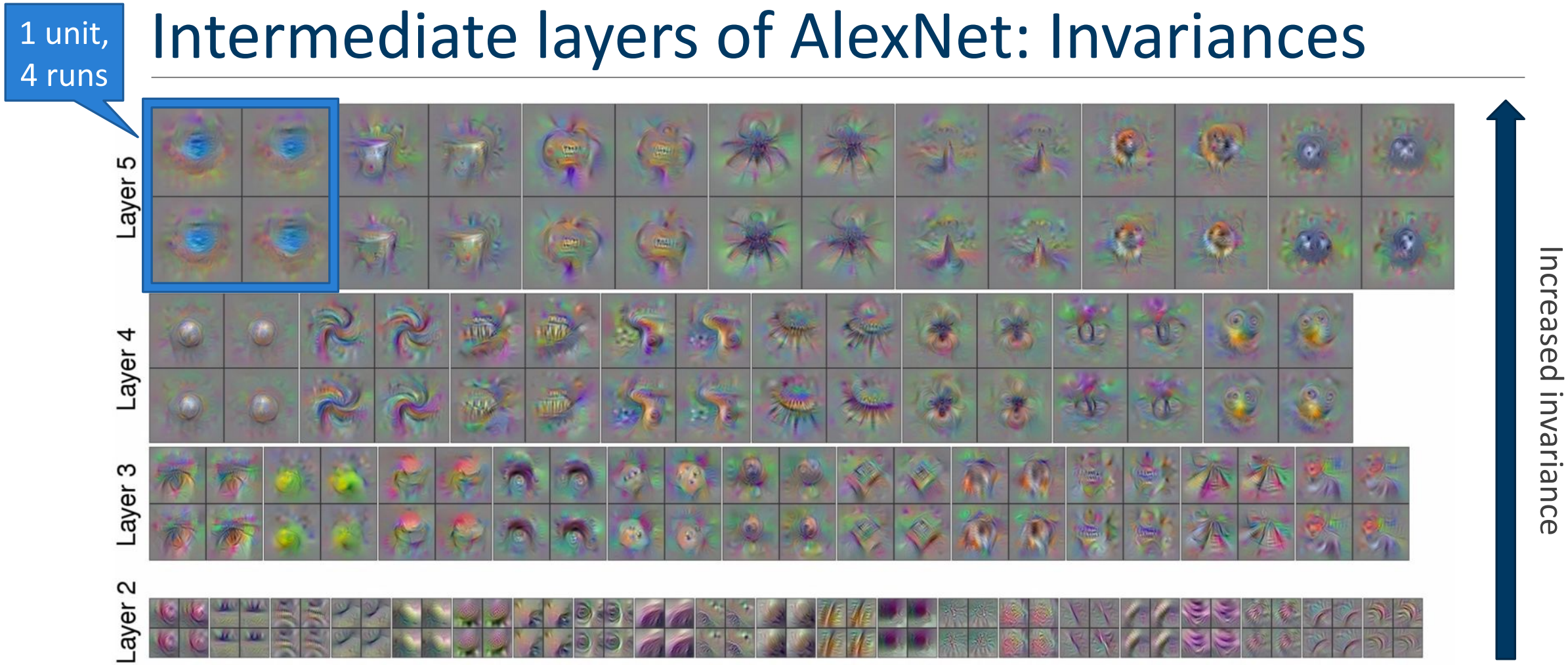


Image source: Nguyen et al, "Synthesizing the preferred inputs for neurons in neural networks via deep generator networks," NeurIPS 2016

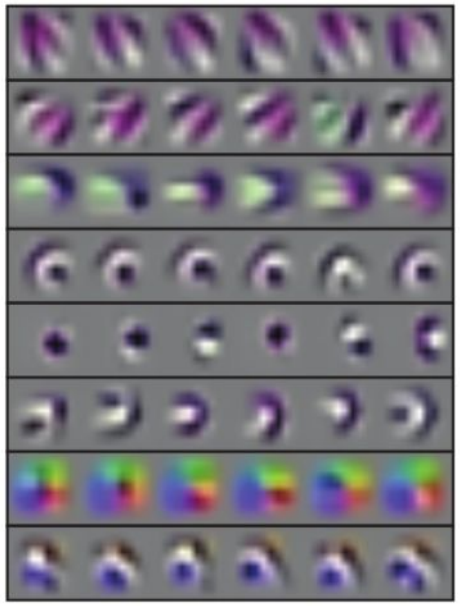
Intermediate layers of AlexNet: Invariances



Yosinski et al., *ICML Deep Learning Workshop* 2015

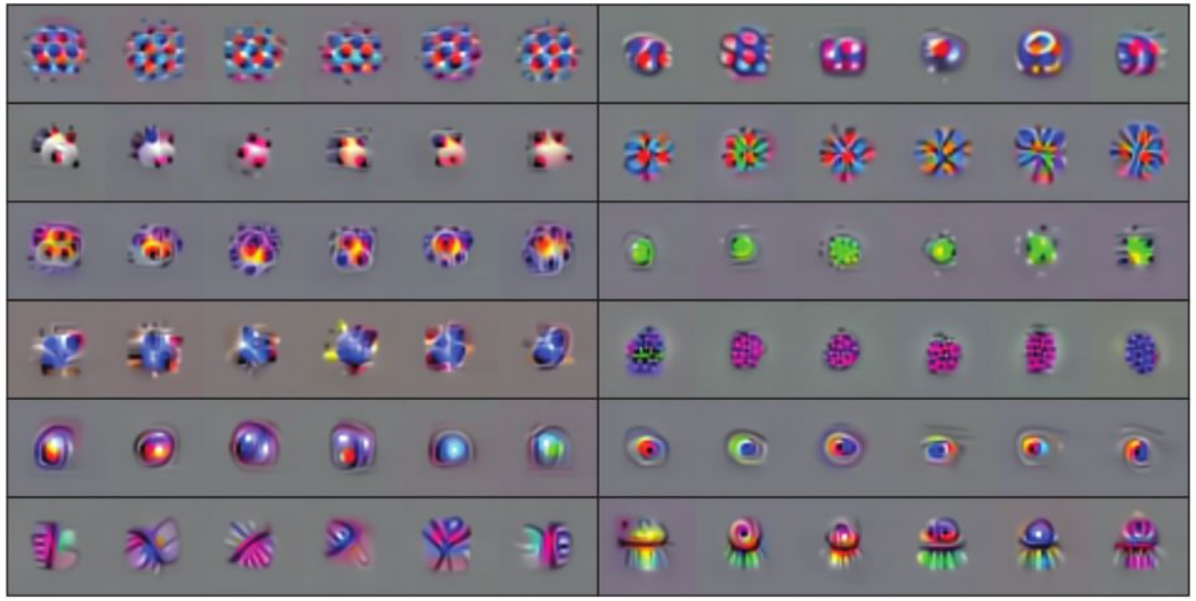
Intermediate layers of VGG-19: Invariances

conv2_1



Increased invariance

conv3_3



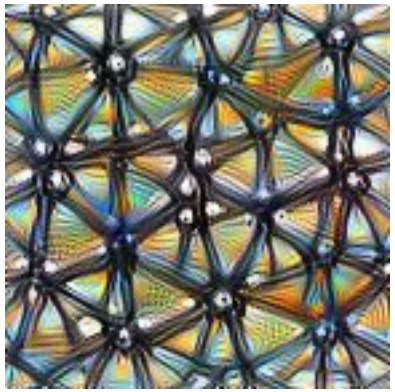
Early: selective

Intermediate/late: Increased invariance

Optimization target here: obtain diverse set of images that maximize overall activity

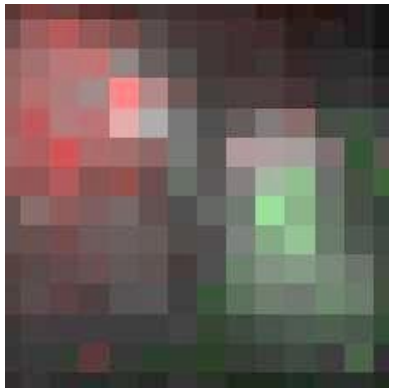
Cadena et al., *ECCV* 2018

How do CNNs form decisions?



What features do individual units “look for”?

Feature visualization

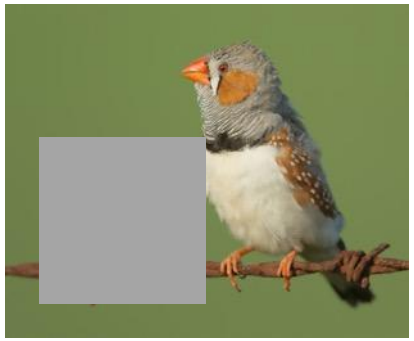


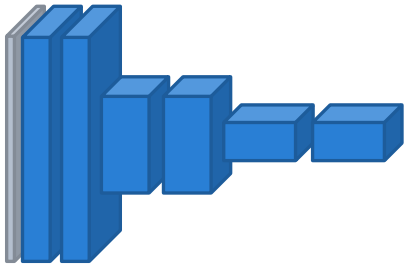
What image features are responsible for the network’s decision?

Attribution

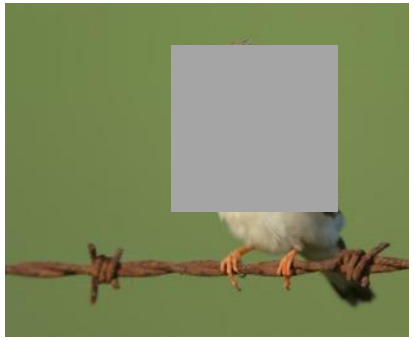
Image source: Olah et al. (2017) https://distill.pub/2017/feature-visualization/

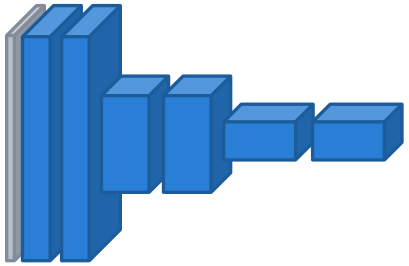
Which pixels matter? Attribution via occlusion





P(zebra finch)
= 0.98





P(zebra finch)
= 0.3

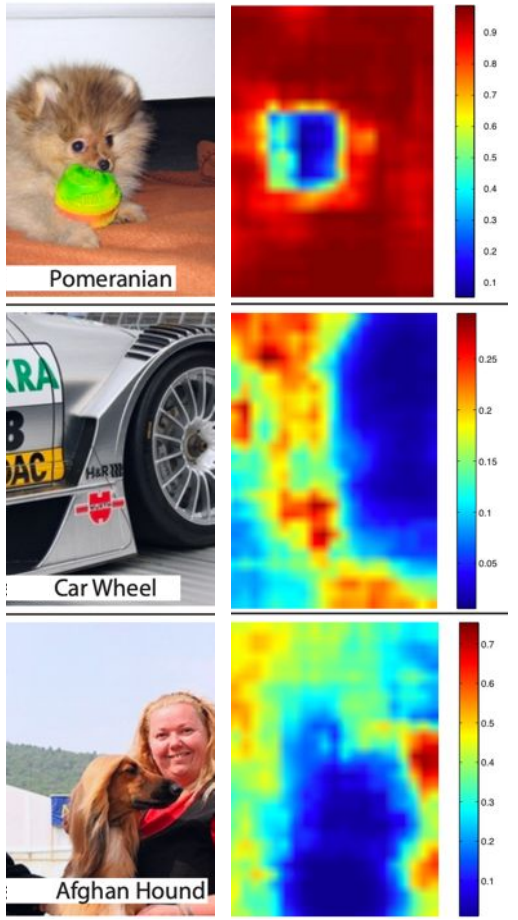


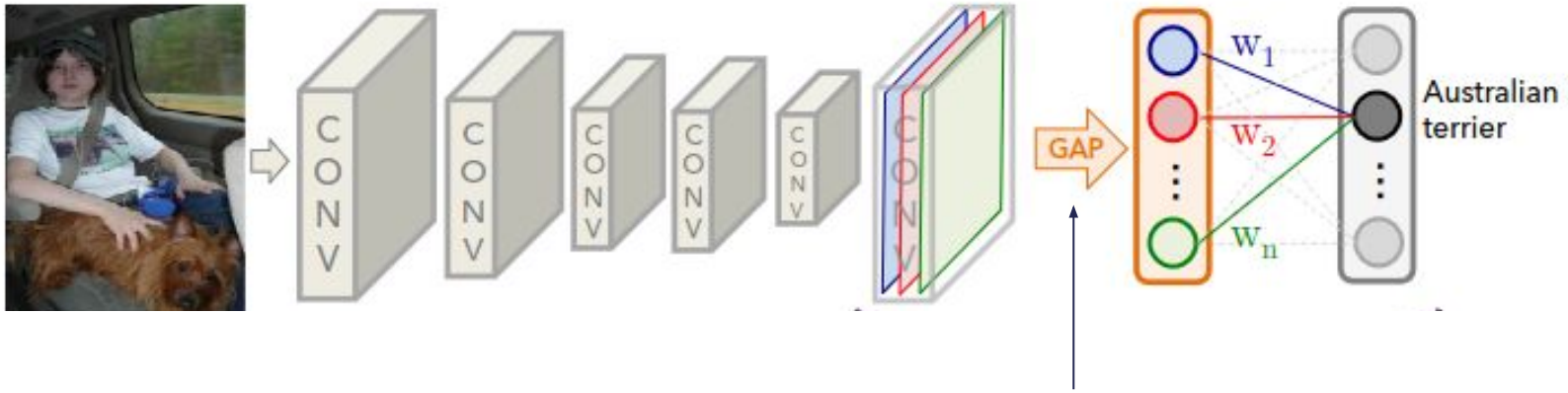
Image source: https://en.wikipedia.org/wiki/Zebra_finch

Class Activation Maps (CAM)

https://arxiv.org/abs/1512.04150

Learn importance of channels in last conv layer for classes.

Fully connected layer -> channel weights for each class



Global average pooling -> channel to scalar

Class Activation Maps (CAM)

https://arxiv.org/abs/1512.04150

Learn importance of channels in last conv layer for classes.

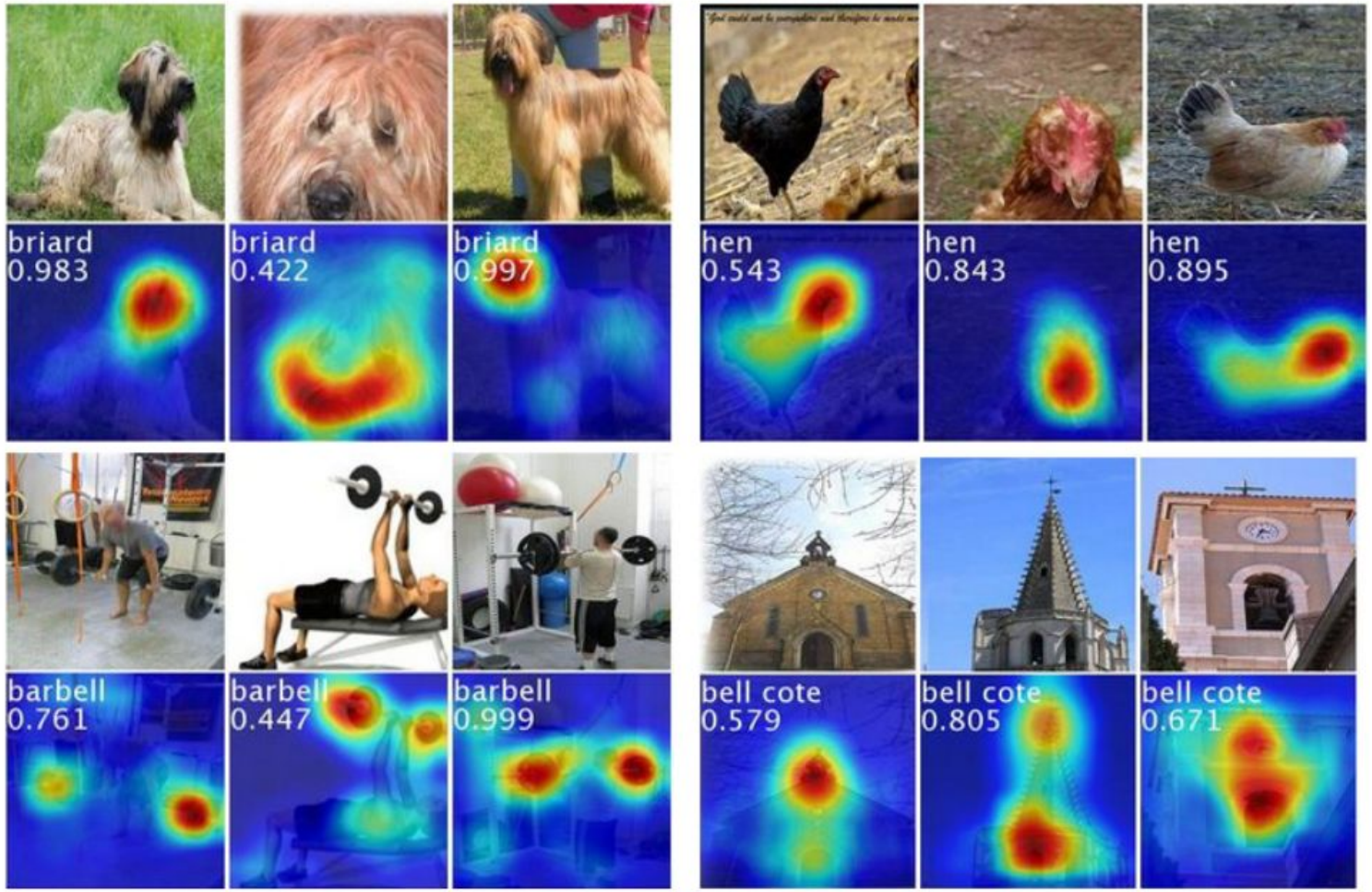
Apply learned class weights to activations of last layer and sum.

Class Activation Maps (CAM)

https://arxiv.org/abs/1512.04150

Learn importance of channels in last conv layer for classes.

Apply learned class weights to activations of last layer and sum.



Class Activation Maps (CAM)

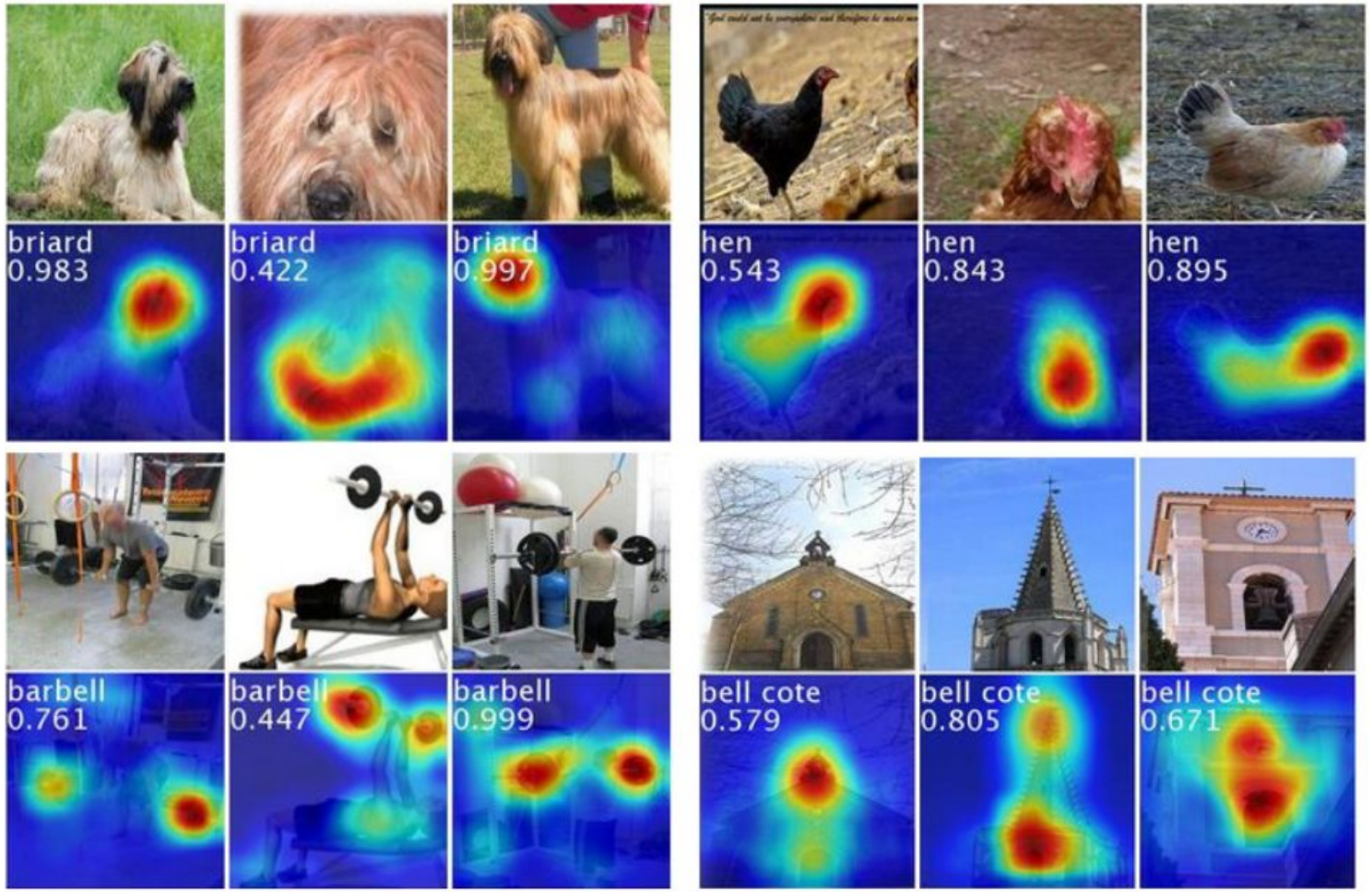
https://arxiv.org/abs/1512.04150

Learn importance of channels in last conv layer for classes.

Apply learned class weights to activations of last layer and sum.

Disadvantage: train new layer

- Additional cost
- Does not use original fully conn. layers



Grad CAM

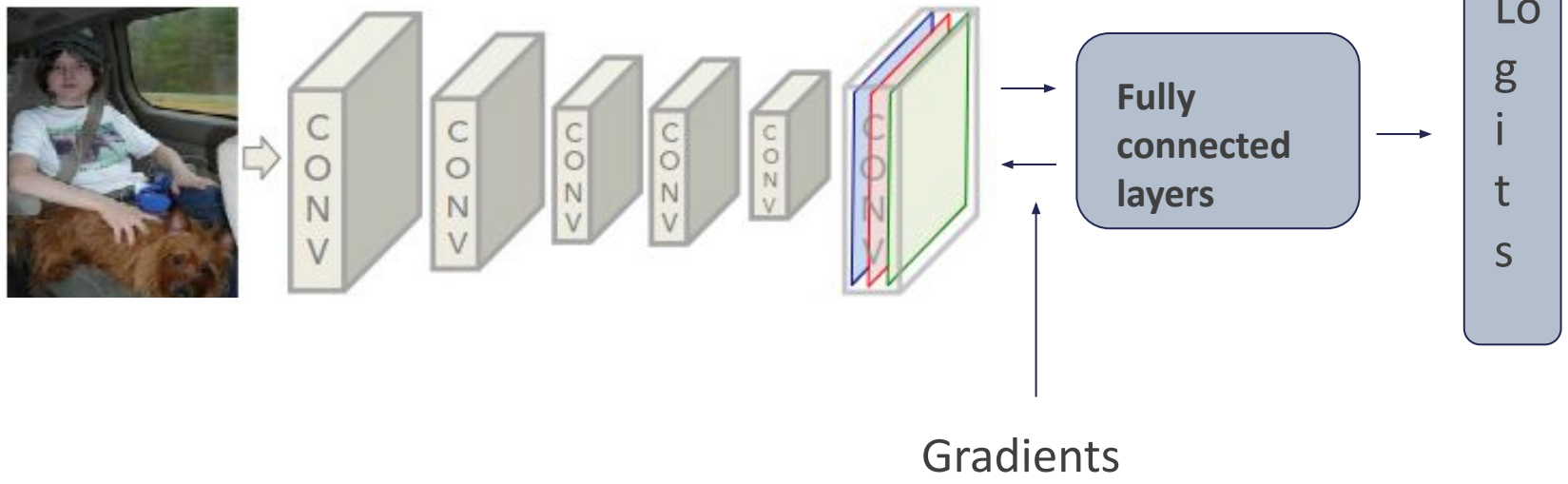
https://arxiv.org/abs/1610.02391

Idea: use gradients w.r.t. classes as weights:

class

$$w_k^c = \sum_i \sum_j \frac{\partial y^c}{\partial A_{ij}^k}$$

channel



Grad CAM

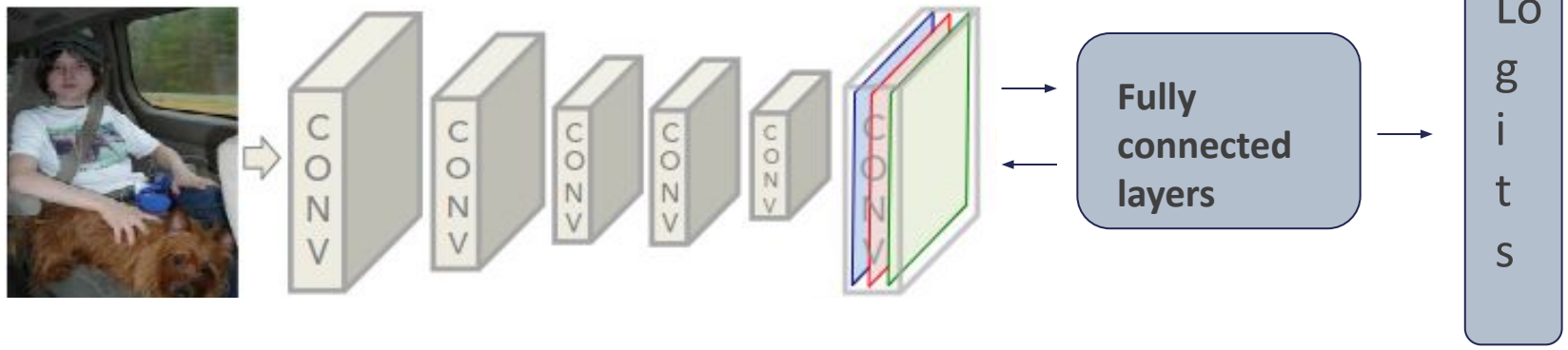
https://arxiv.org/abs/1610.02391

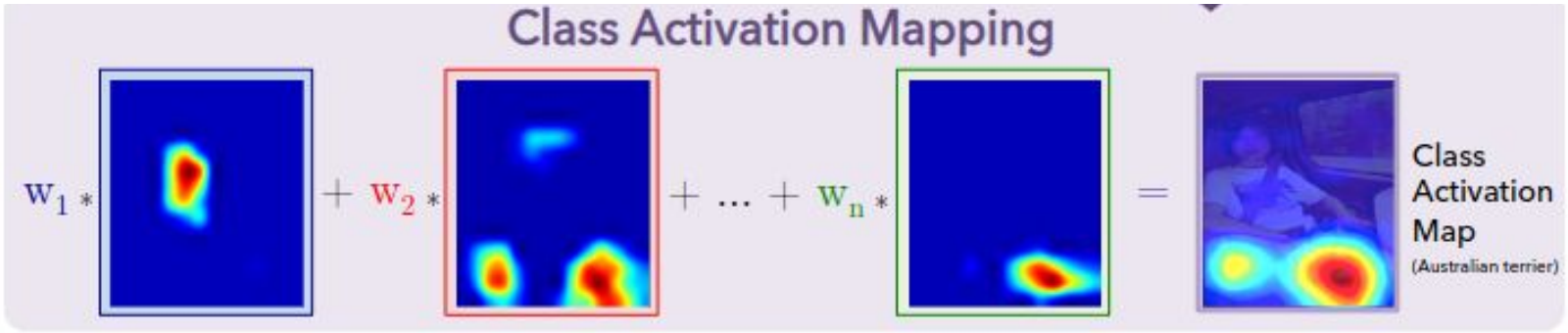
Idea: use gradients w.r.t. classes as weights:

class

$$w_k{}^c = \sum_i \sum_j \frac{\partial y^c}{\partial A_{ij}{}^k}$$

channel





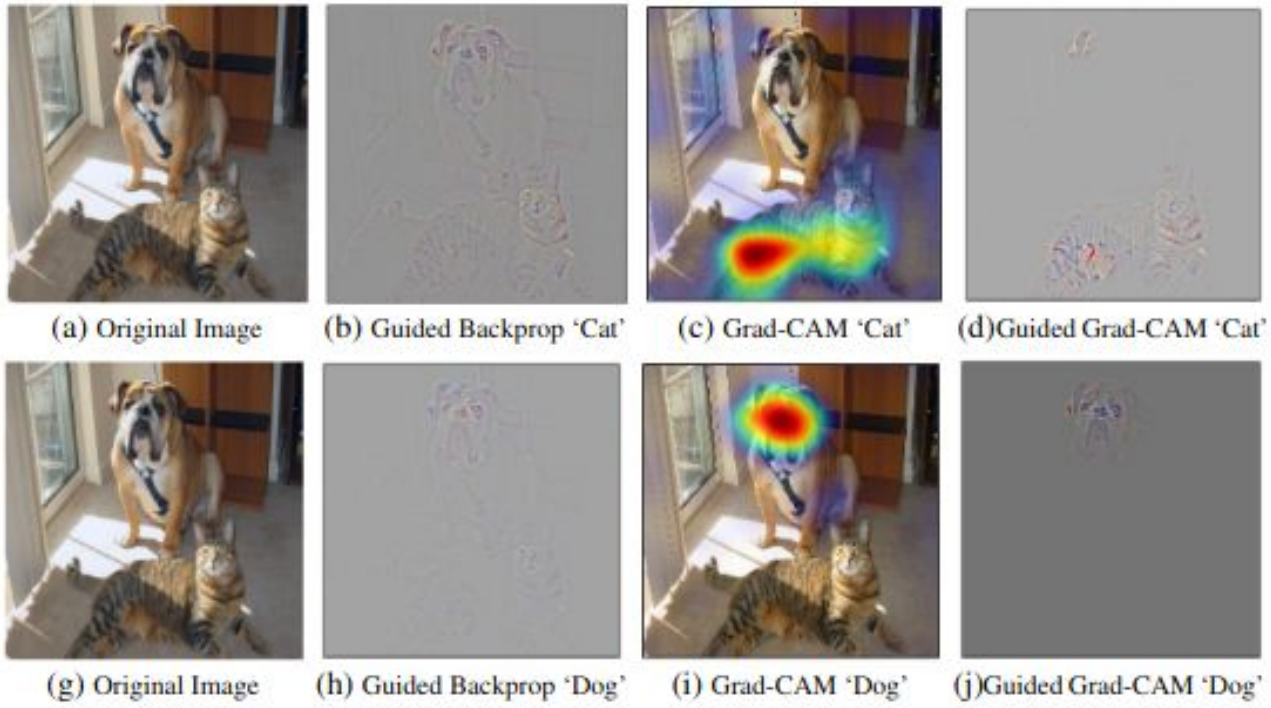
Guided Grad CAM

https://arxiv.org/abs/1610.02391

Can be combined with Guided Backprop (only propagate positive gradients)

Advantes:

- No retraining necessary
- Uses full architecture



Improving GradCAM

https://arxiv.org/abs/1710.11063

Grad CAM++:

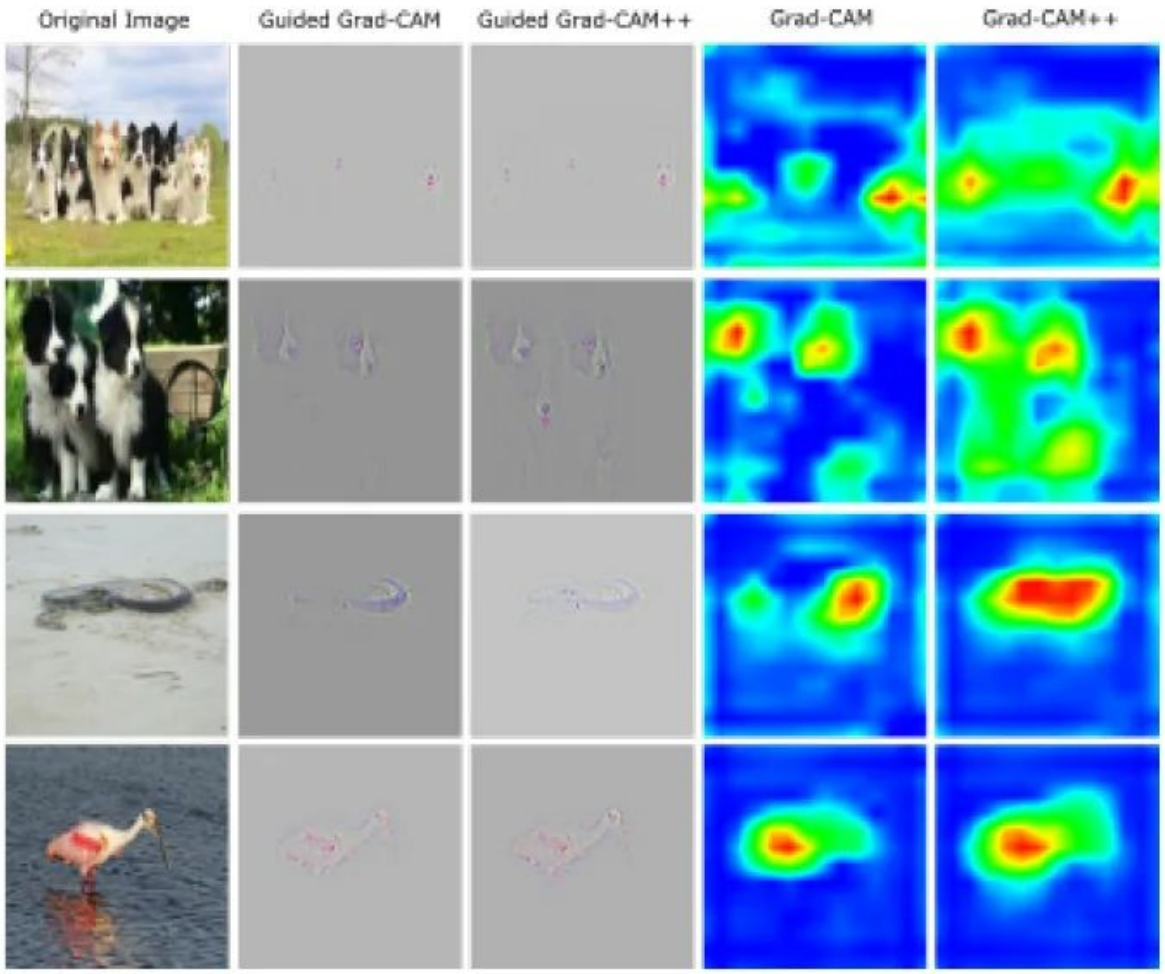
- modify by saliency factor alpha
- better spatial attribution

Grad-CAM++

$$w_k^c = \sum_i \sum_j \alpha_{ij}^{kc} . relu(\frac{\partial Y^c}{\partial A_{ij}^k})$$

Grad-CAM

$$w_k^c = \frac{1}{Z} \sum_i \sum_j \frac{\partial Y^c}{\partial A_{ij}^k}$$



Integrated Gradients

https://arxiv.org/abs/1703.01365

Instead of computing gradients w.r.t. class:

- Transform baseline (black image) into image
- Sum gradients along the path (approx. path integral)

$$IntegratedGrads_i(x) ::= (x_i - x_i') \times \int_{\alpha=0}^{1} \frac{\partial F(x' + \alpha \times (x - x'))}{\partial x_i} d\alpha$$

$$IntegratedGrads_i^{approx}(x) ::= (x_i - x_i') \times \sum_{k=1}^{m} \frac{\partial F(x' + \frac{k}{m} \times (x - x'))}{\partial x_i} \times \frac{1}{m}$$

Integrated Gradients

https://arxiv.org/abs/1703.01365

Instead of computing gradients w.r.t. class:

- Transform baseline (black image) into image
- Sum gradients along the path (approx. path integral)

$$IntegratedGrads_i^{approx}(x) ::= (x_i - x_i') \times \sum_{k=1}^{m} \frac{\partial F(x' + \frac{k}{m} \times (x - x'))}{\partial x_i} \times \frac{1}{m}$$

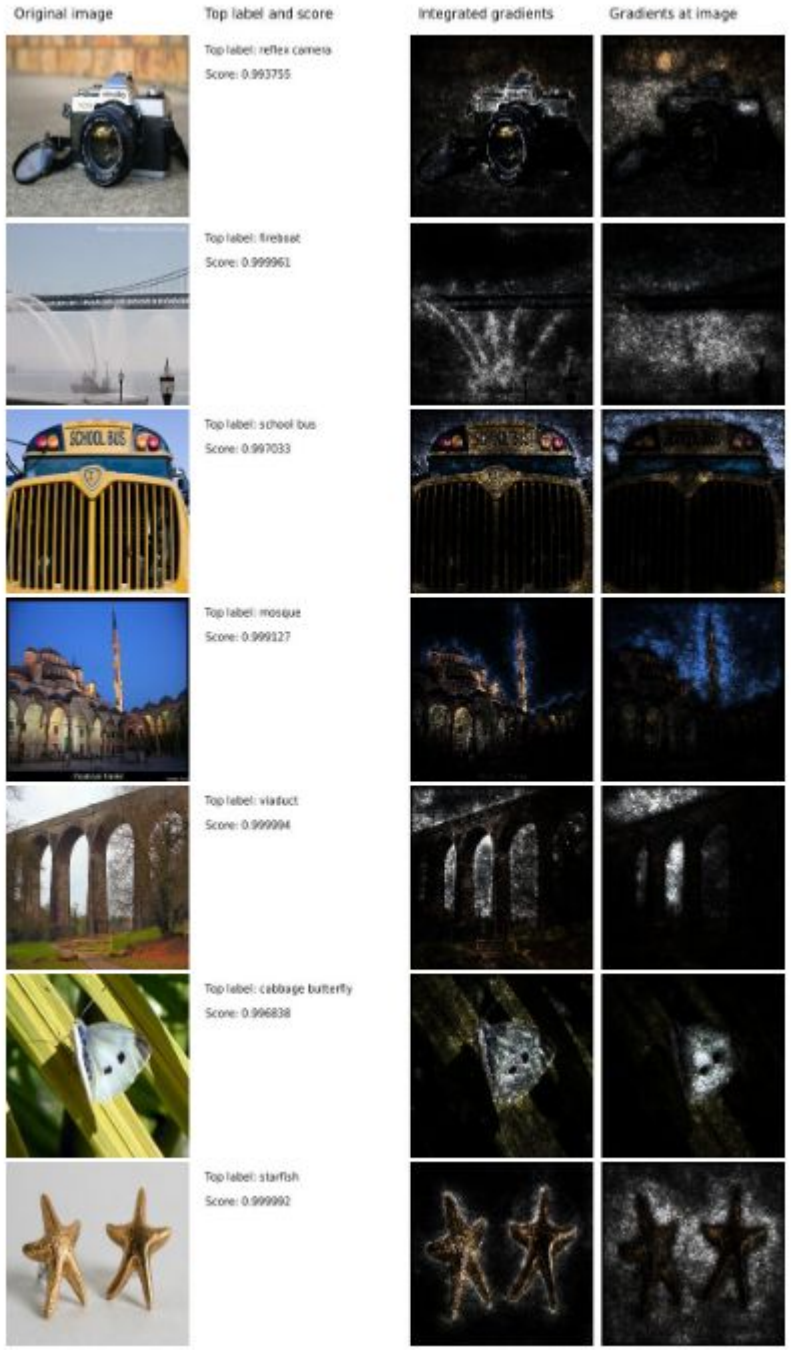
Interpolation baseline to image

Integrated Gradients

Instead of computing gradients w.r.t. class:

- Transform baseline (black image) into image
- Sum gradients along the path (approx. path integral)

$$IntegratedGrads_i^{approx}(x) ::= (x_i - x_i') \times \sum_{k=1}^{m} \frac{\partial F(x' + \frac{k}{m} \times (x - x'))}{\partial x_i} \times \frac{1}{m}$$



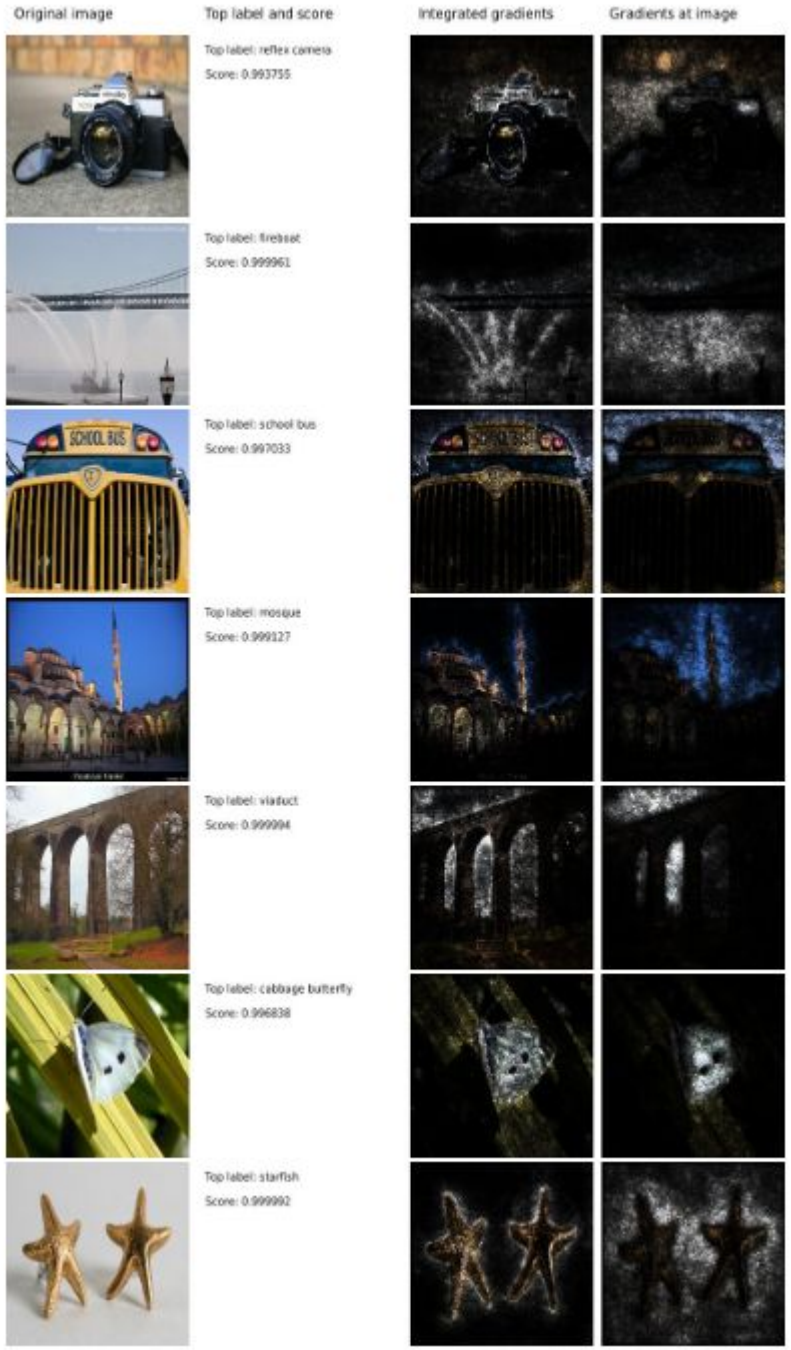
Integrated Gradients

Instead of computing gradients w.r.t. class:

- Transform baseline (black image) into image
- Sum gradients along the path (approx. path integral)

$$IntegratedGrads_i^{approx}(x) ::= (x_i - x_i') \times \sum_{k=1}^{m} \frac{\partial F(x' + \frac{k}{m} \times (x - x'))}{\partial x_i} \times \frac{1}{m}$$

Intuition: Strong gradients represent importance for the decision.



Interpretable Machine Learning

Visualizing and understanding deep neural networks is a very active branch of research!

“Interpretable Machine Learning”

- Visualize and understand trained networks (this lecture)
- Design models and training algorithms that are easier to interpret

Further reading:

- distill.pub articles by Chris Olah
  - https://distill.pub/2017/feature-visualization/ (<- images shown in the lecture)
  - https://distill.pub/2018/building-blocks/
- Interpretable Machine Learning book by Christoph Molnar
  - https://christophm.github.io/interpretable-ml-book/

Neural style transfer

TEXTURE SYNTHESIS AND PERCEPTUAL LOSS FUNCTIONS

History of art – Tübingen


Vincent van Gogh (1889)


William Turner (1805)


Pablo Picasso (1910)


Edvard Munch (1893)


Wassily Kandinsky (1913)

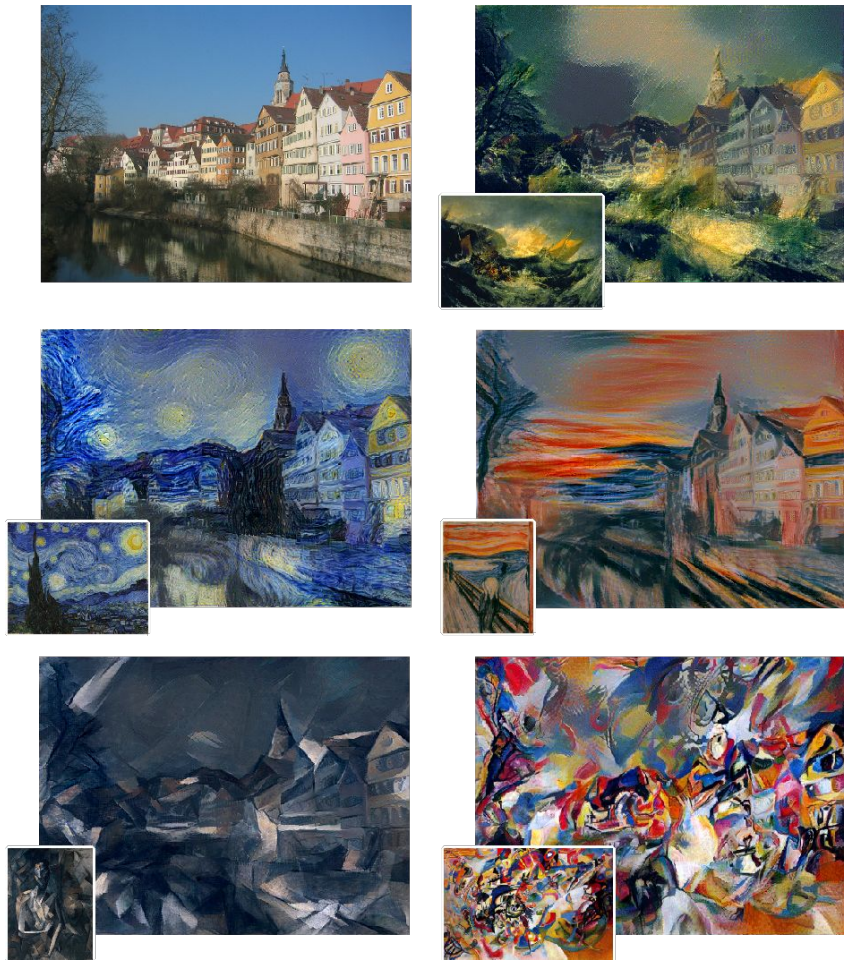
A Neural Algorithm of Artistic Style

Image Style Transfer Using Convolutional Neural Networks

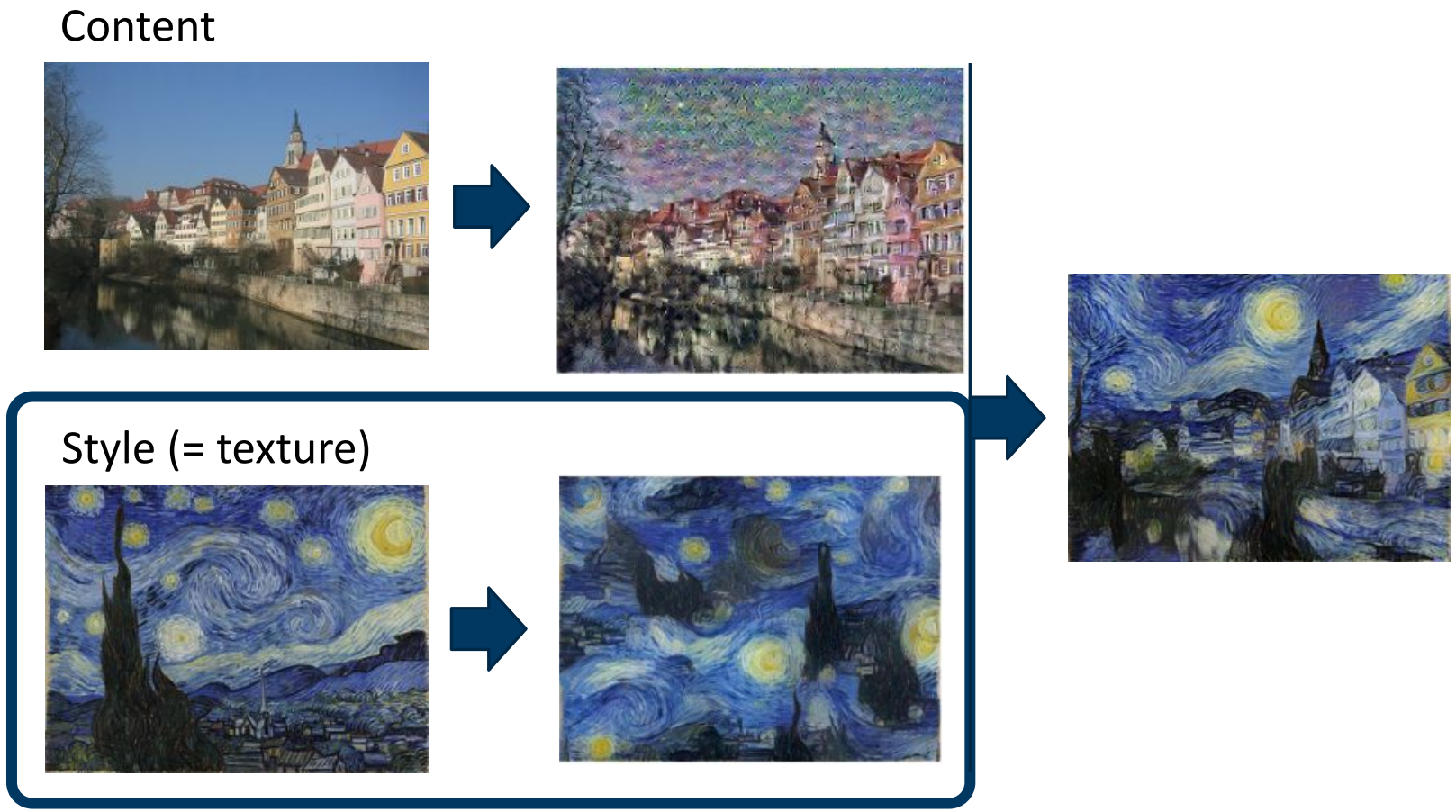
Leon A. Gatys
Centre for Integrative Neuroscience, University of Tübingen, Germany
Bernstein Center for Computational Neuroscience, Tübingen, Germany
Graduate School of Neural Information Processing, University of Tübingen, Germany
leon.gatys@bethgelab.org

Alexander S. Ecker
Centre for Integrative Neuroscience, University of Tübingen, Germany
Bernstein Center for Computational Neuroscience, Tübingen, Germany
Max Planck Institute for Biological Cybernetics, Tübingen, Germany
Baylor College of Medicine, Houston, TX, USA

Matthias Bethge
Centre for Integrative Neuroscience, University of Tübingen, Germany
Bernstein Center for Computational Neuroscience, Tübingen, Germany
Max Planck Institute for Biological Cybernetics, Tübingen, Germany



Neural Style Transfer



Gatys, Ecker Bethge, CVPR 2016

A brief history of parametric texture modeling

Julesz (1962)

- Conjecture: N^{th} order summary statistics determine texture perception

Portilla & Simoncelli (2000)

- Summary statistics of features from early visual system (steerable pyramid)

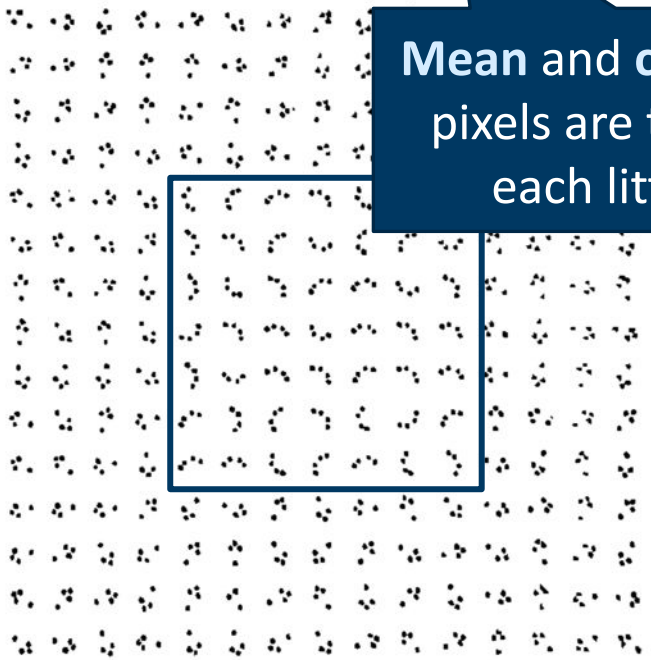
Gatys, Ecker Bethge (NIPS 2015)

- Summary statistics of features from CNNs trained on ImageNet

Julesz' conjecture (1962)

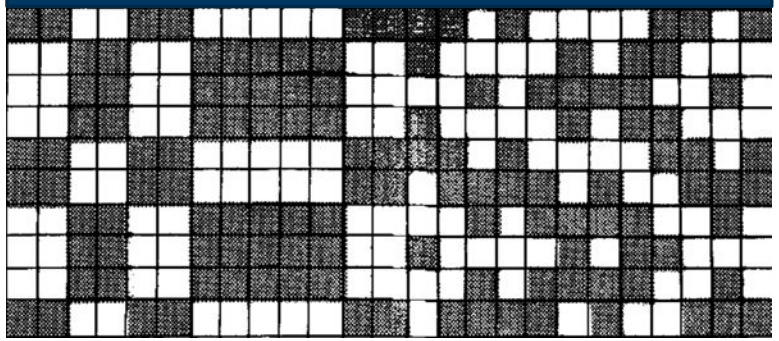
N^{th} order summary statistics determine texture perception.

Uniform 2nd order statistics (power spectrum)



Mean and **covariance** of pixels are the same in each little patch

Uniform 3rd-order statistics



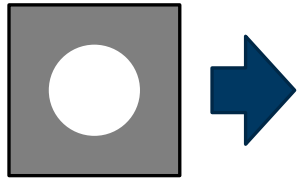
Mean, covariance + **triplet correlations** are the same in left and right half

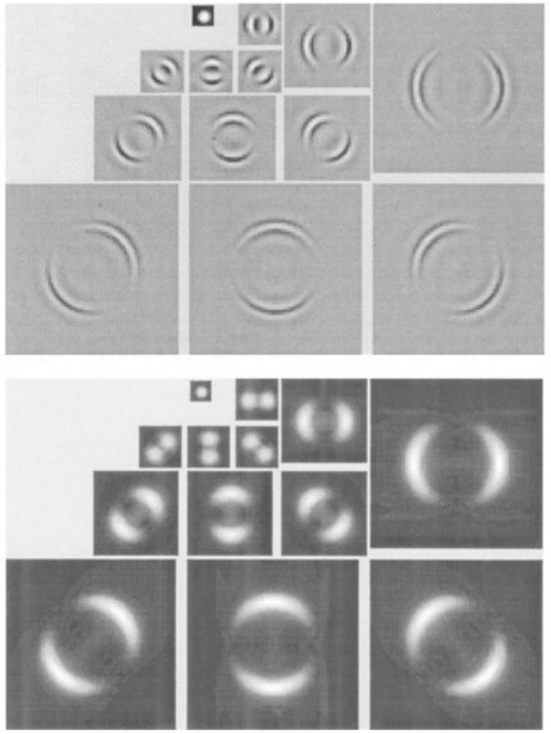
Julesz 1962, Julesz et al. 1978

Portilla & Simoncelli texture model

Steerable pyramid
(Simoncelli & Freeman 1995)

Image





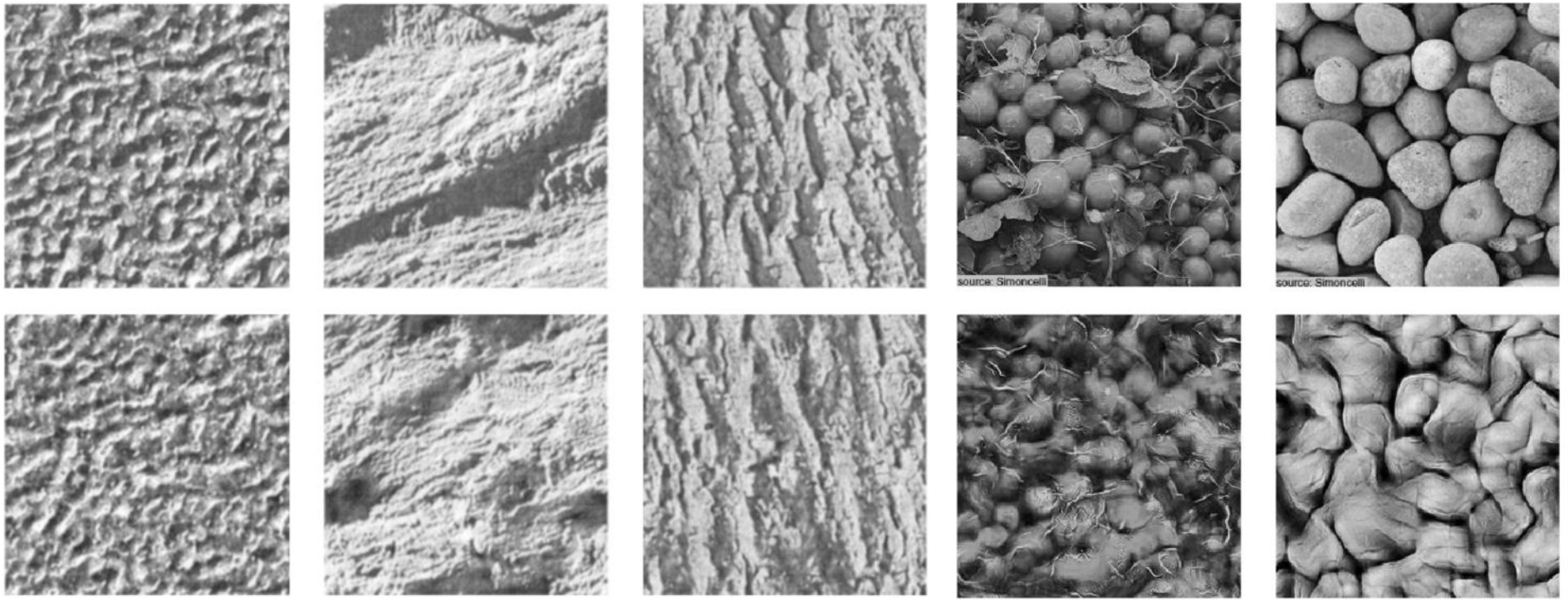
Summary statistics

- Mean
- Variance
- Correlations across
  - Space
  - Features

Portilla & Simoncelli (2000)

Portilla & Simoncelli: results

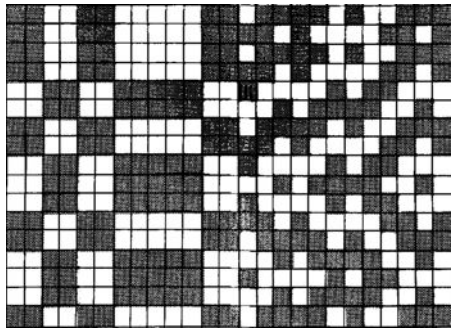
Original images



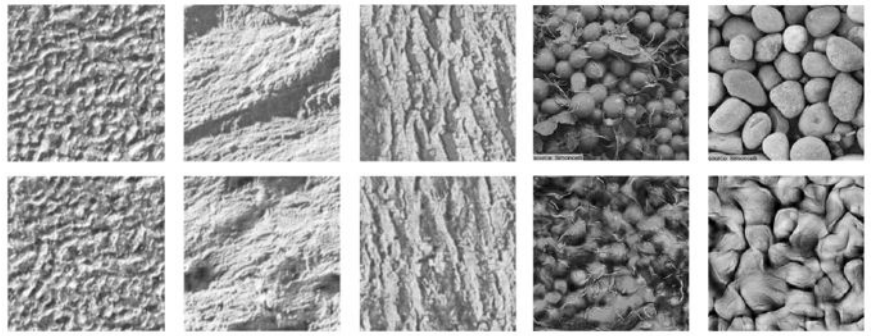
Synthesized by Portilla & Simoncelli

Portilla & Simoncelli (2000)

Texture models: summary statistics



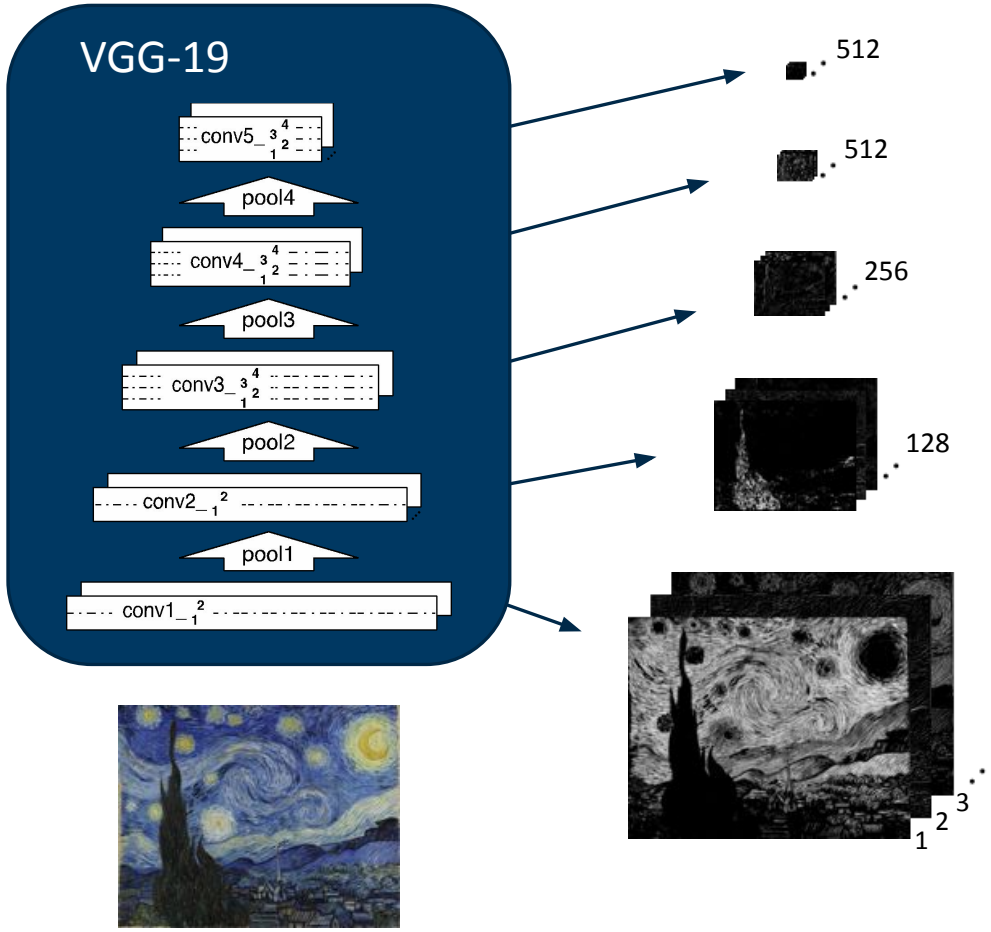
Julesz (1962):
N^{th} order pixel stats



Portilla & Simoncelli (2000):
V1 summary statistics = Steerable Pyramid

Gatys, Ecker, Bethge (2015): "Entire visual pathway" -> CNN features

CNN: Multiscale nonlinear filter bank



Summary statistic:
feature correlations

Gatys, Ecker Bethge, *NeurIPS* 2015

Gram matrix computation

conv1_1 feature maps

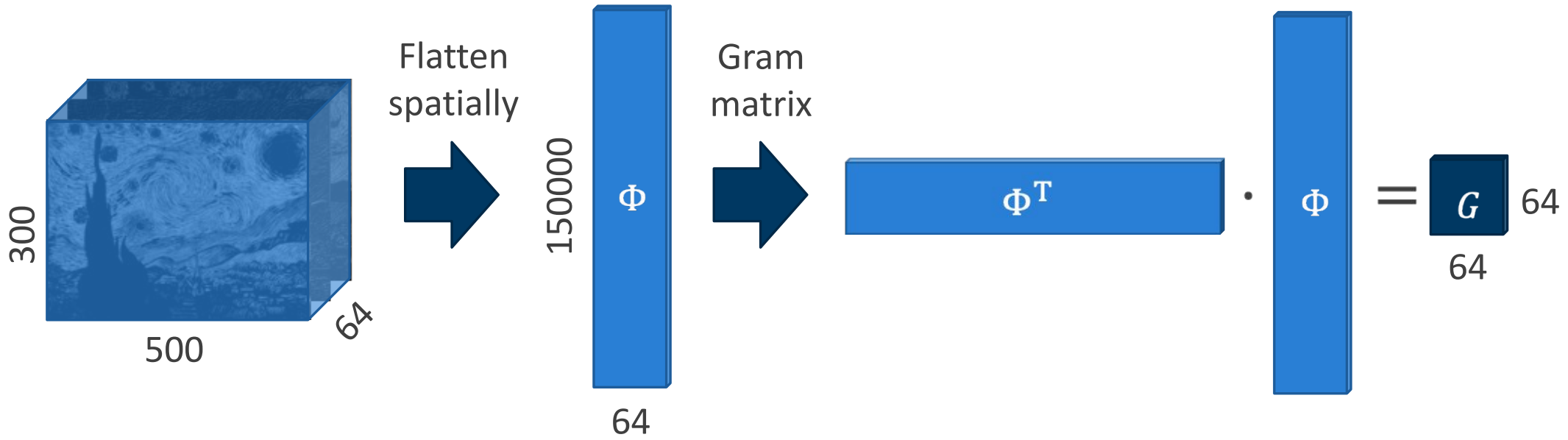
$300 \times 500 \times 64$

Flattened spatially

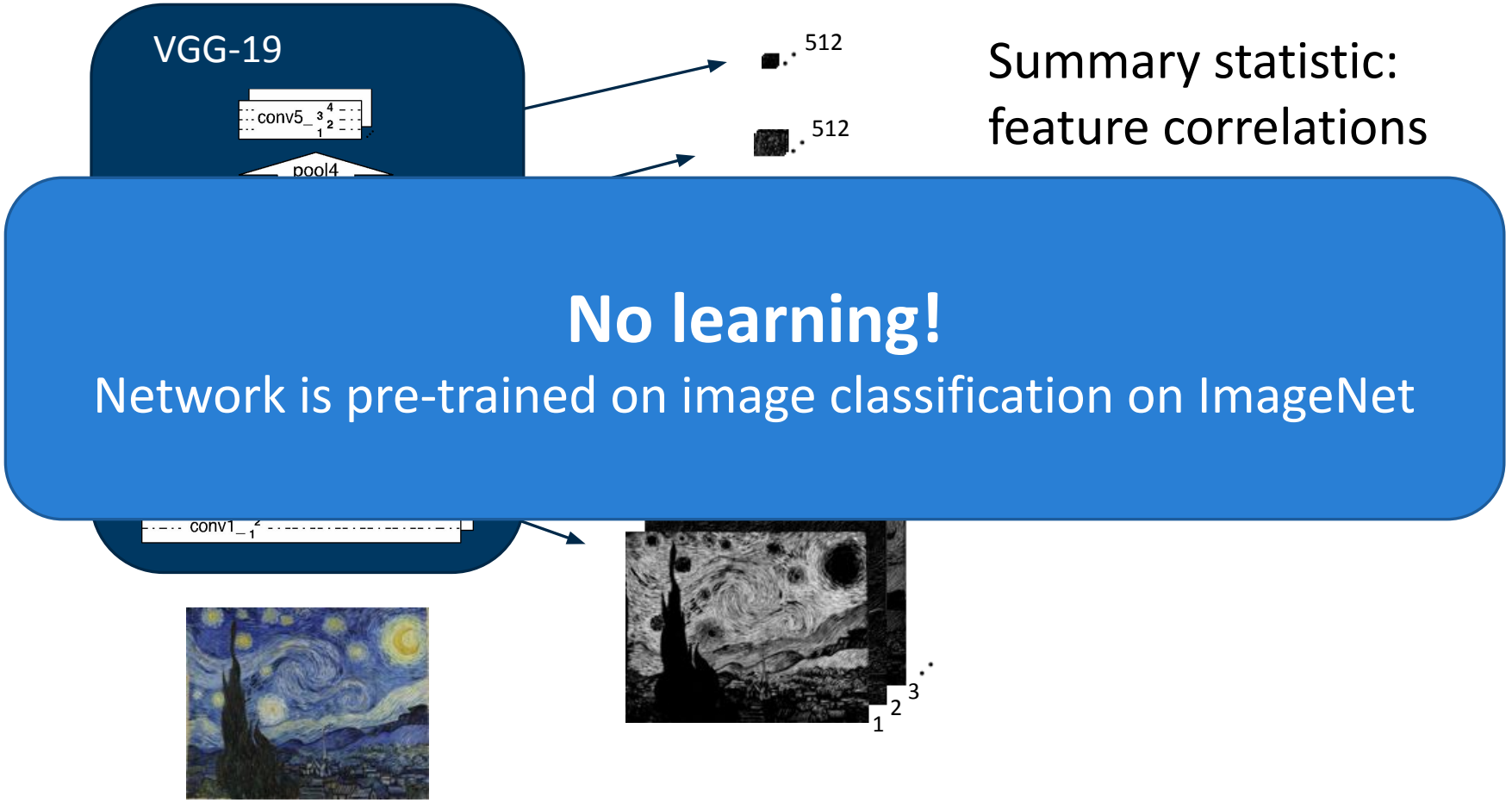
150000×64

Gram matrix

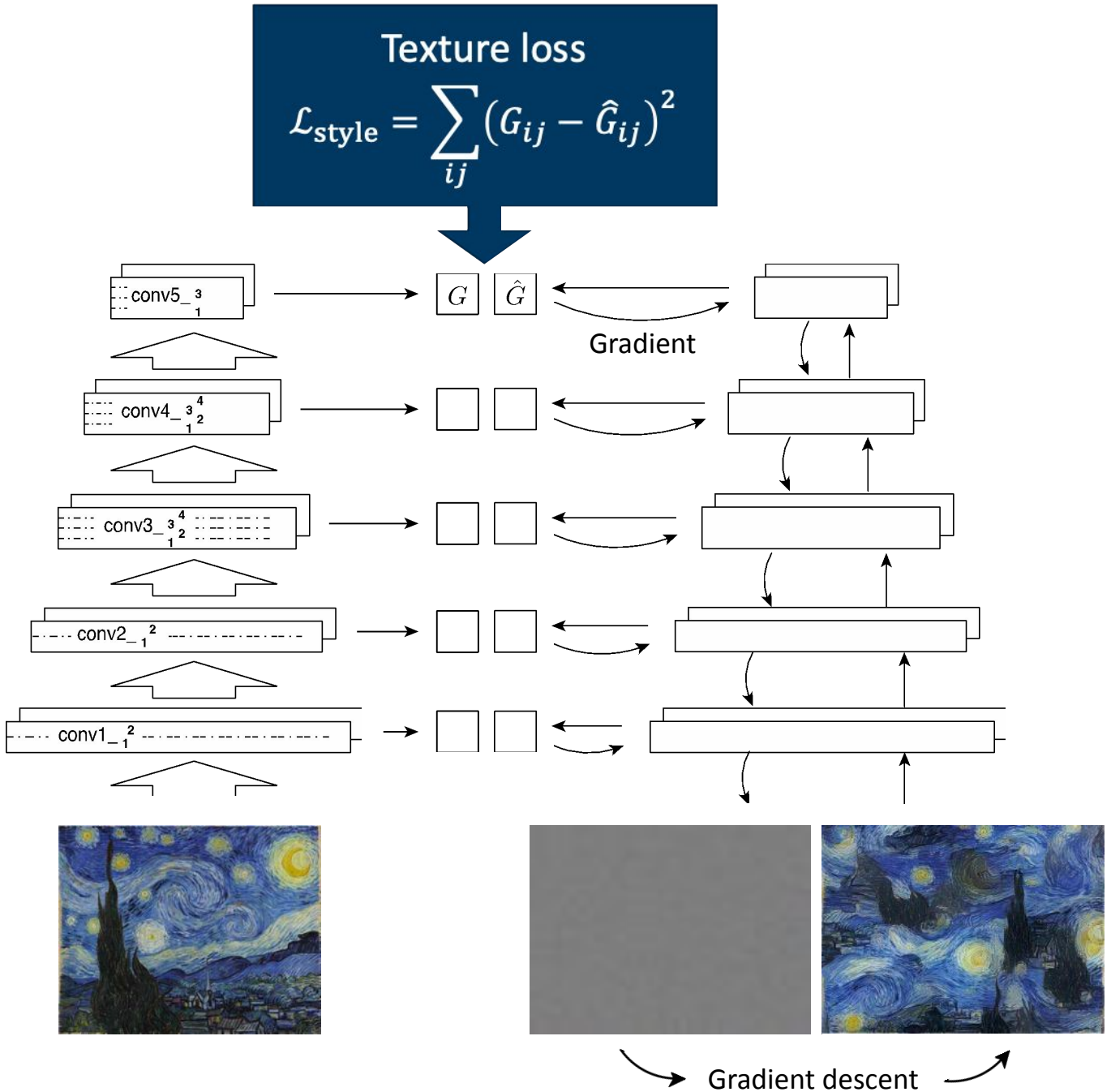
64×64



CNN: Multiscale nonlinear filter bank



Gatys, Ecker Bethge, *NeurIPS* 2015



Gatys, Ecker Bethge, *NeurIPS* 2015

Textures based on deep features

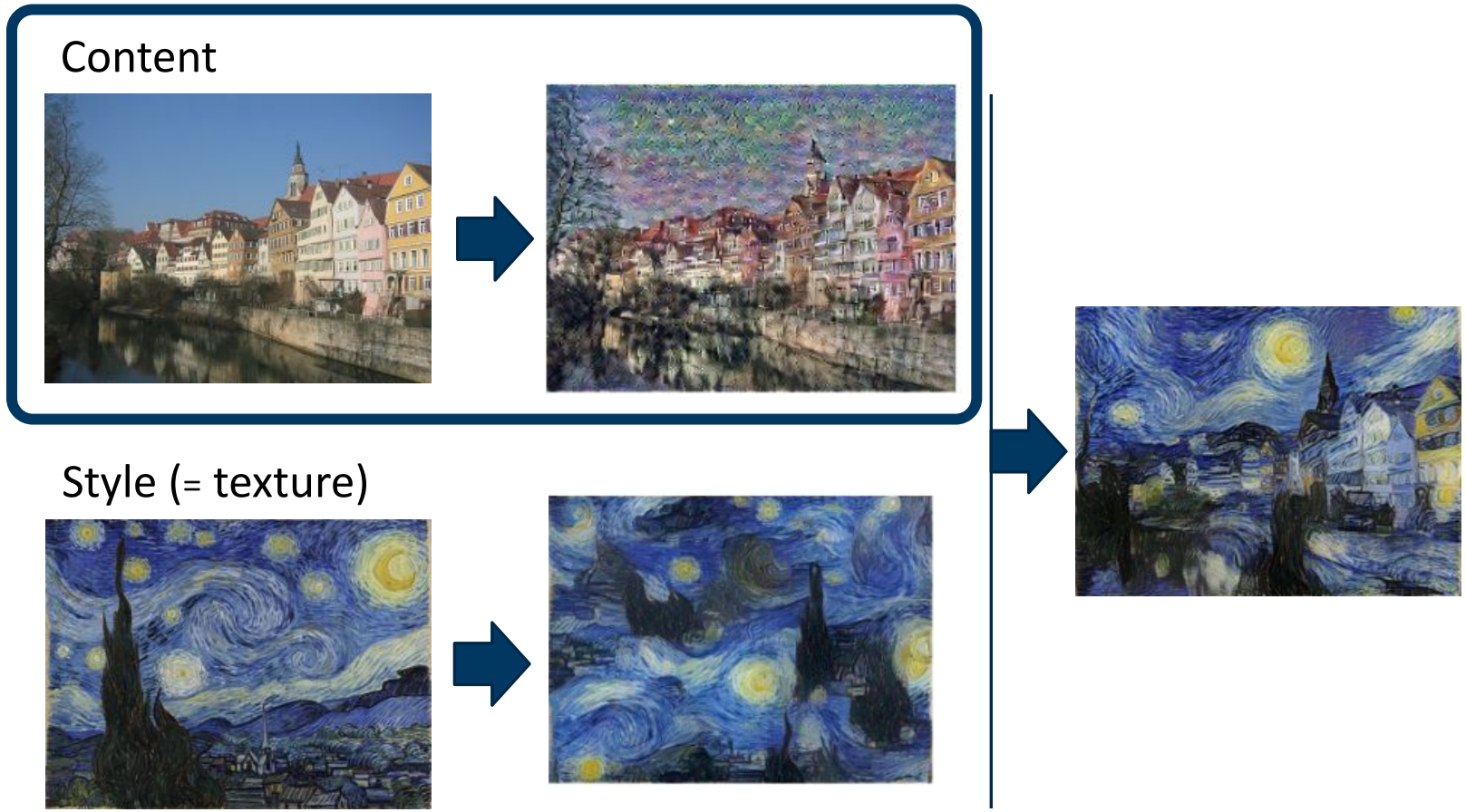
CNN-based model

Original image

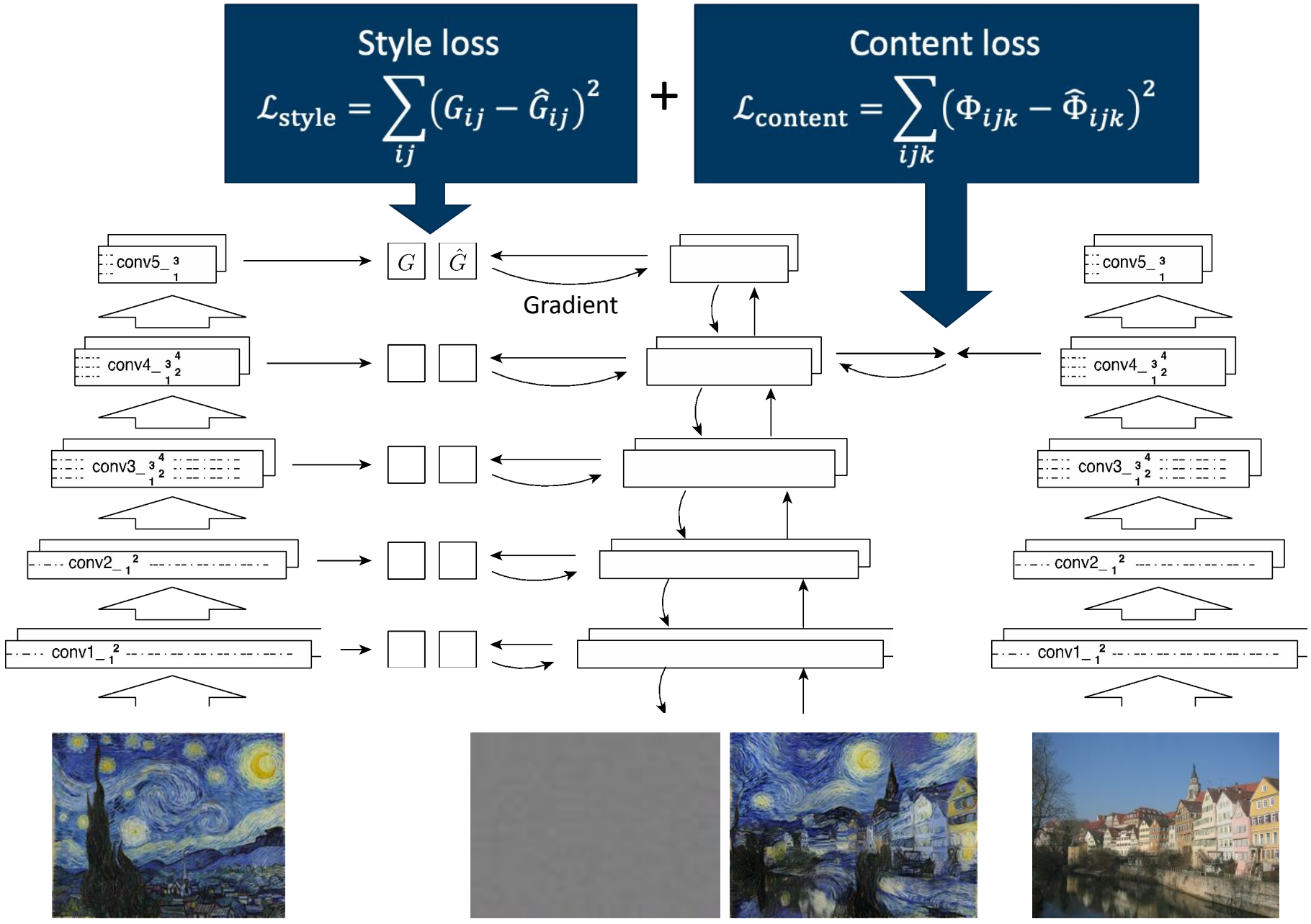
Portilla & Simoncelli

Gatys, Ecker Bethge, *NeurIPS* 2015

Neural Style Transfer



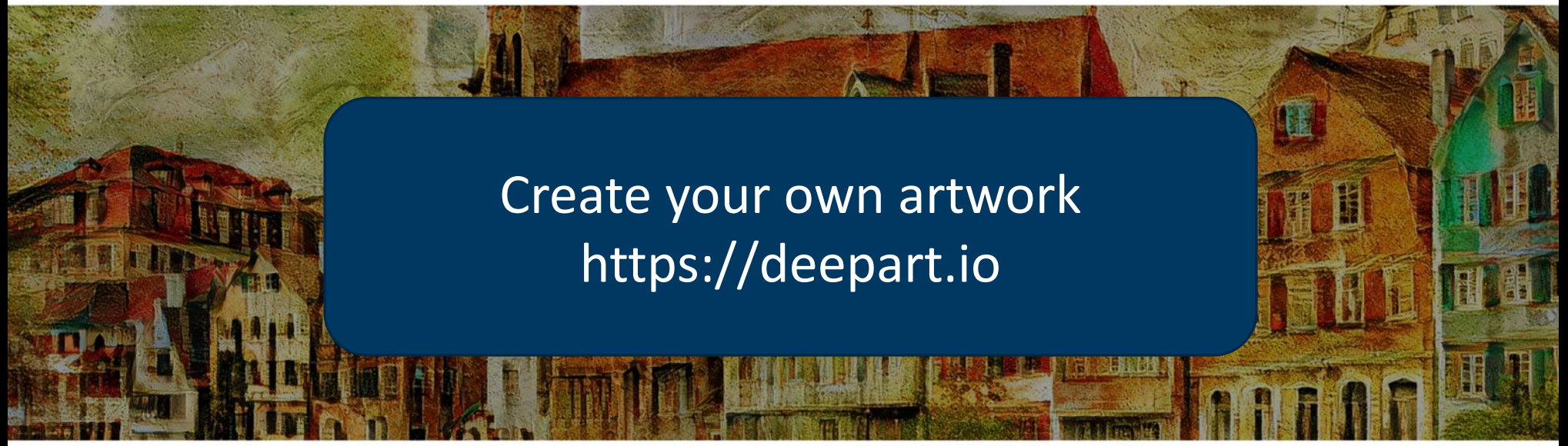
Gatys, Ecker Bethge, *CVPR* 2016



Gatys, Ecker Bethge, *CVPR* 2016

deepart.io

DEEPART.io Latest artworks **CREATE YOUR OWN** Videos Offer About Register Sign in



Create your own artwork
https://deepart.io

TURN ANY PHOTO INTO AN ARTWORK – FOR FREE!

We use an algorithm inspired by the human brain. It uses the stylistic elements of one image to draw the content of another. Get your own artwork in just three steps.

1 Upload photo

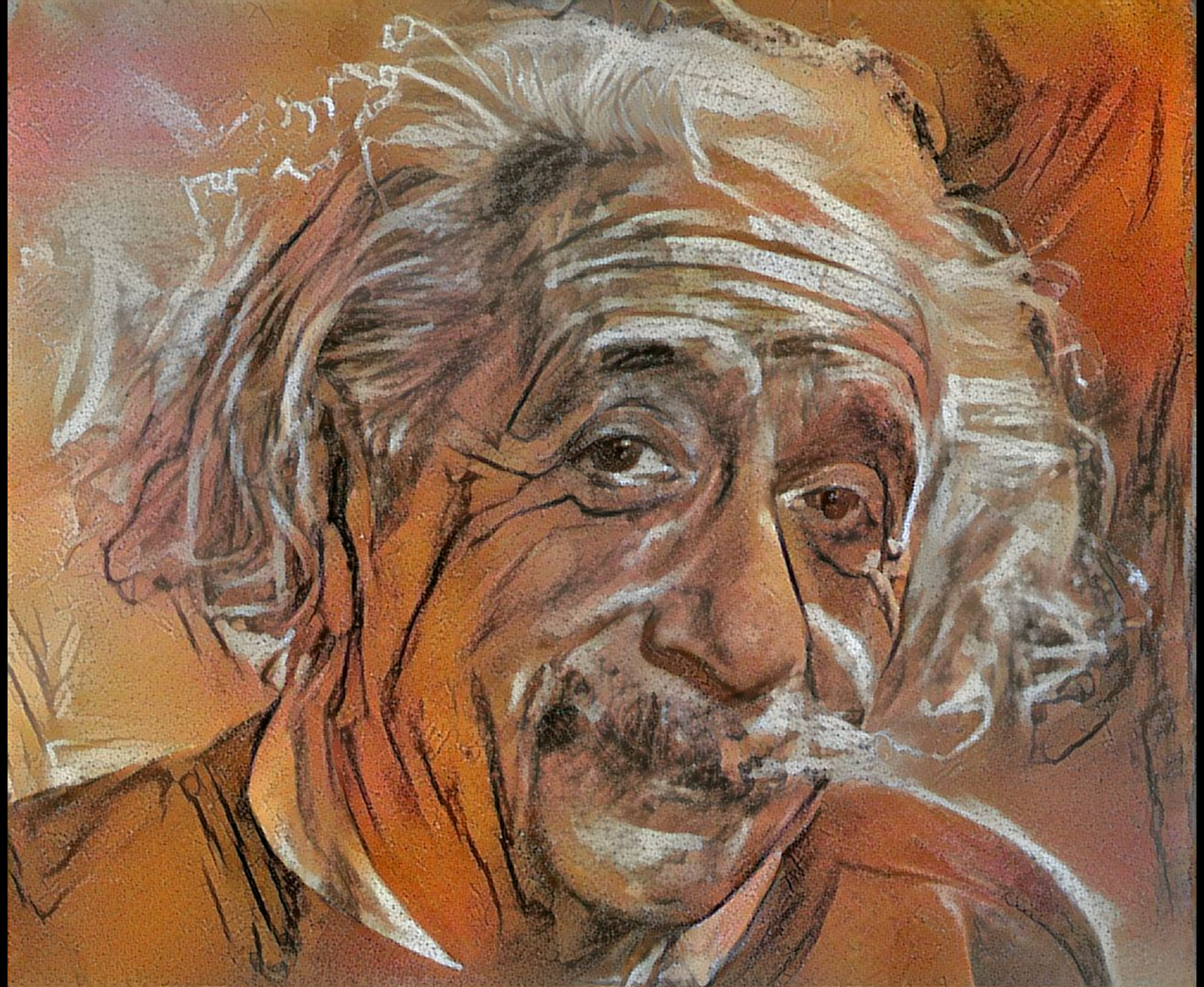
The first picture defines the scene you would like to have painted.

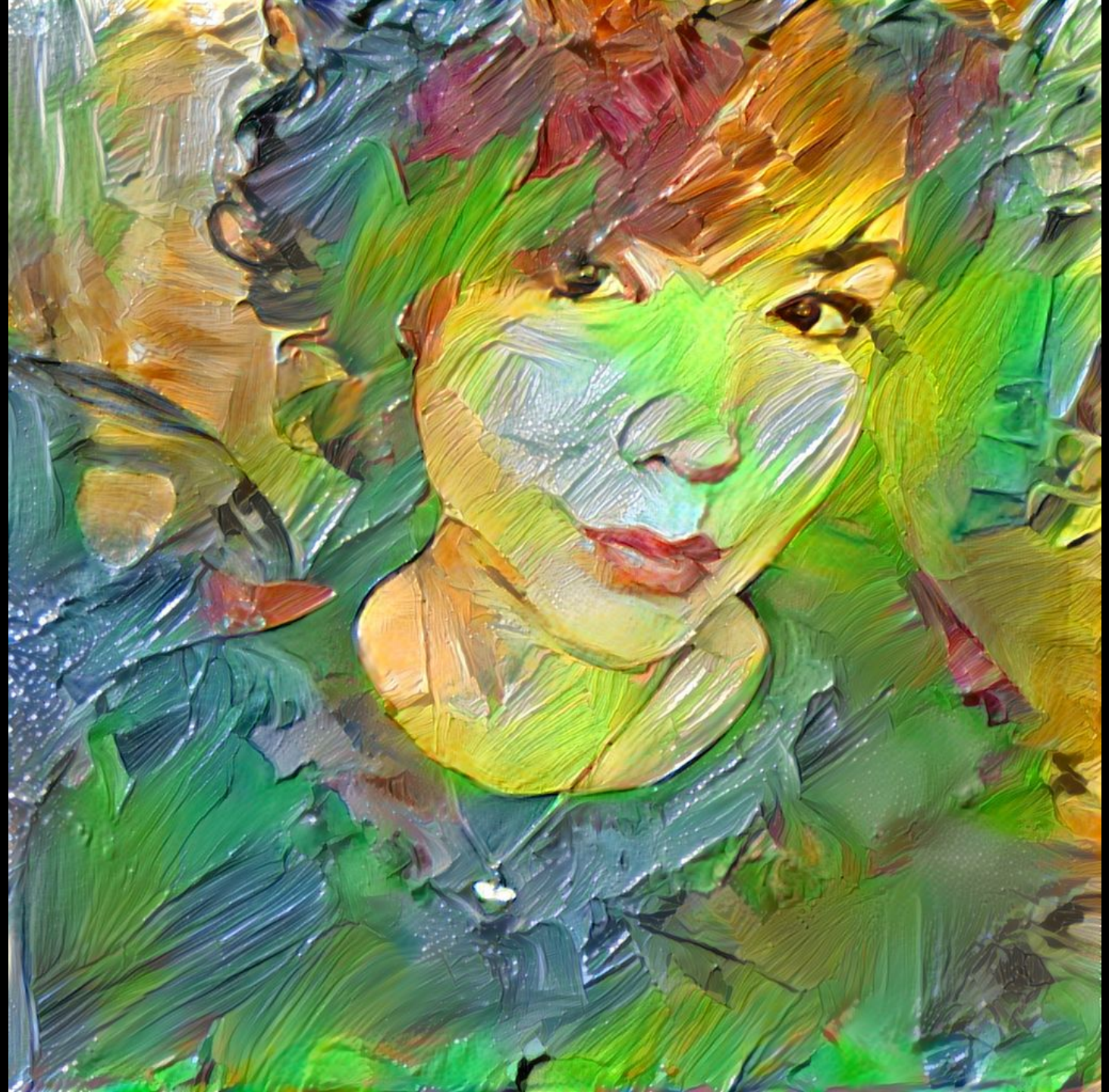
2 Choose style

Choose among predefined styles or upload your own style image.

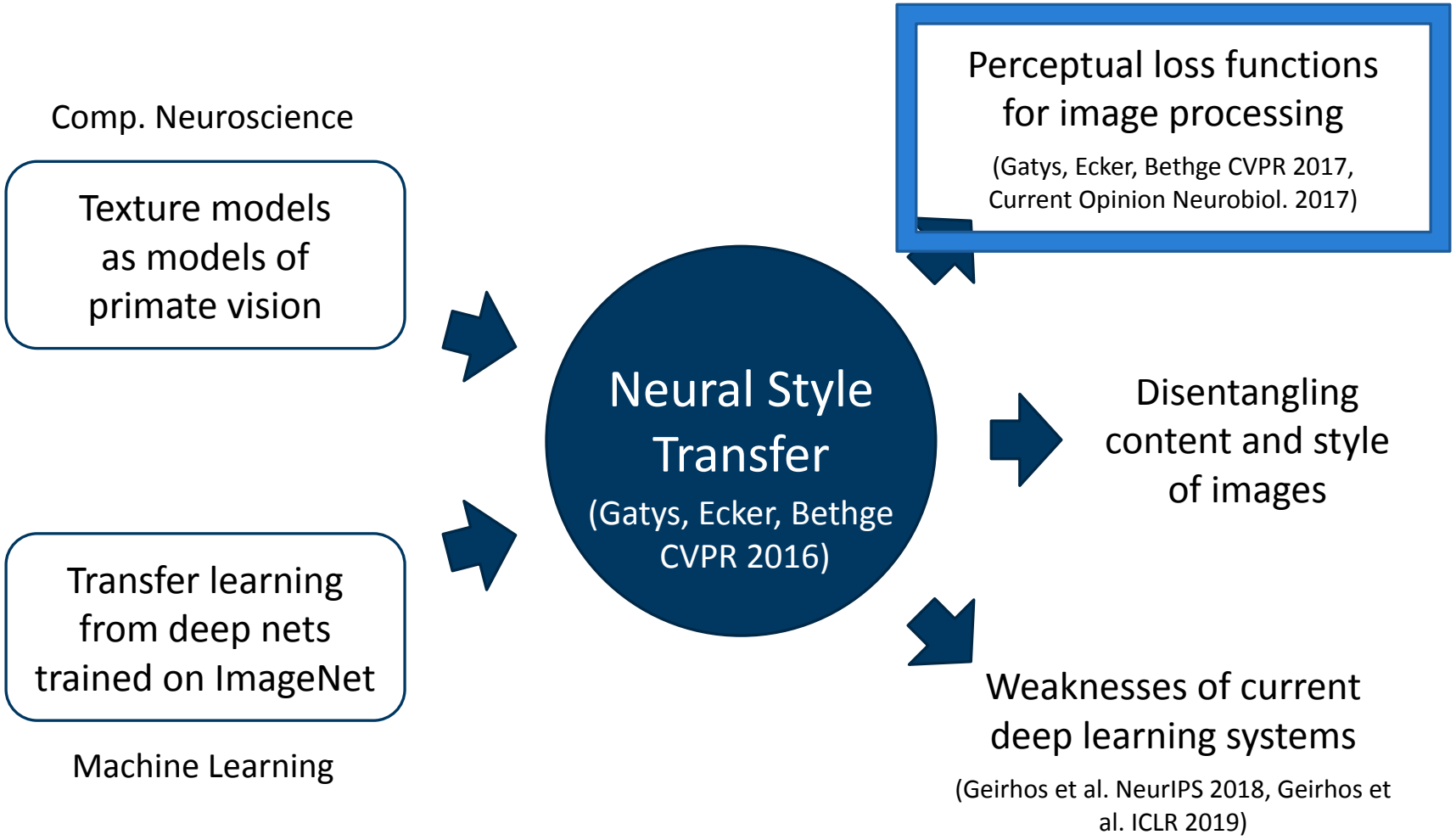
3 Submit

Our servers paint the image for you. You get an email when it's done.



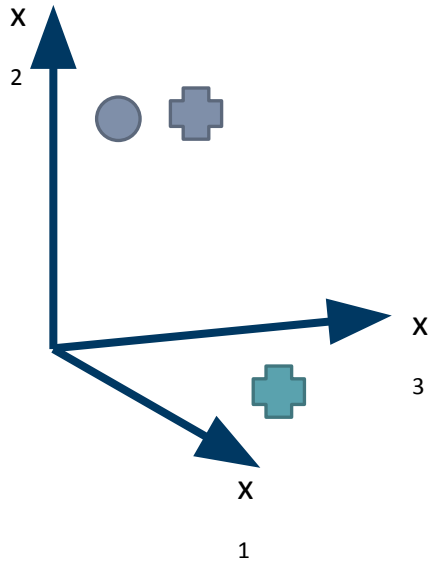


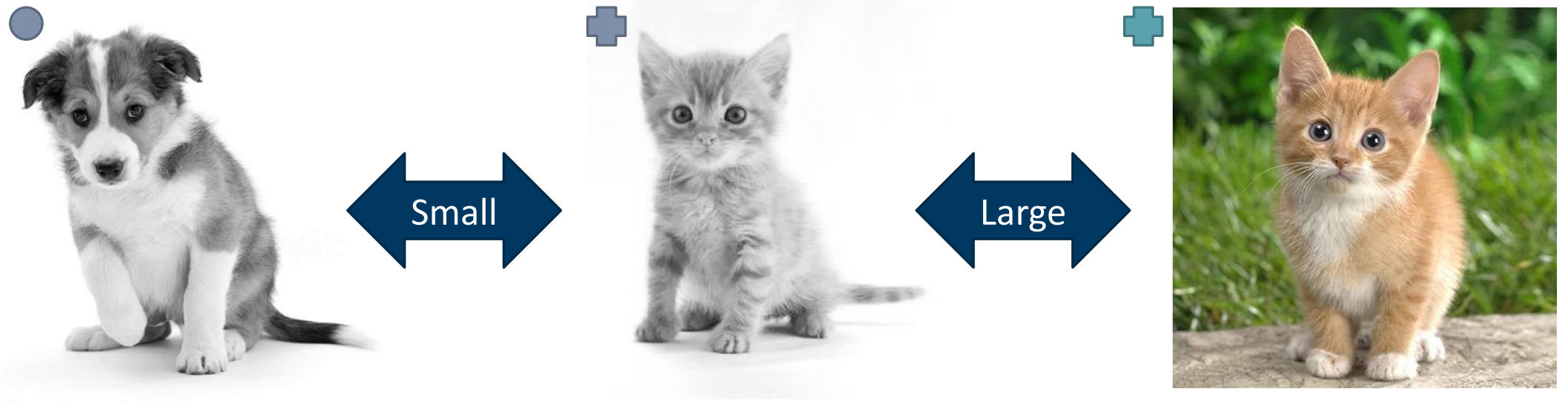
Significance of neural style transfer



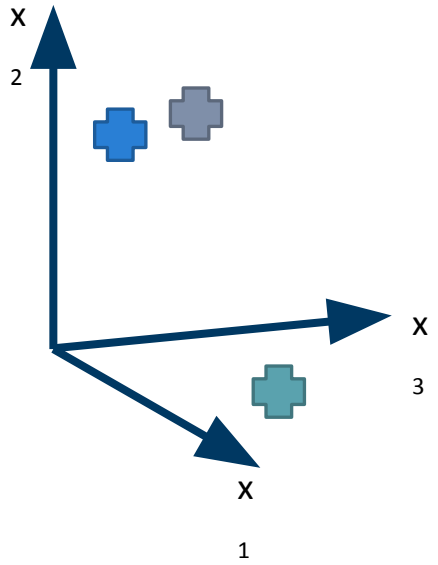
Perceptual loss functions

Euclidean distance in pixel space does not cover semantics of images





Perceptual loss functions



Euclidean distance in pixel space does not cover perceptual differences

Compute distances in high-level feature space of deep neural network trained on ImageNet

Blurred | Original | Shifted by 20 pixels

Perceptual loss applications

Recap: style transfer

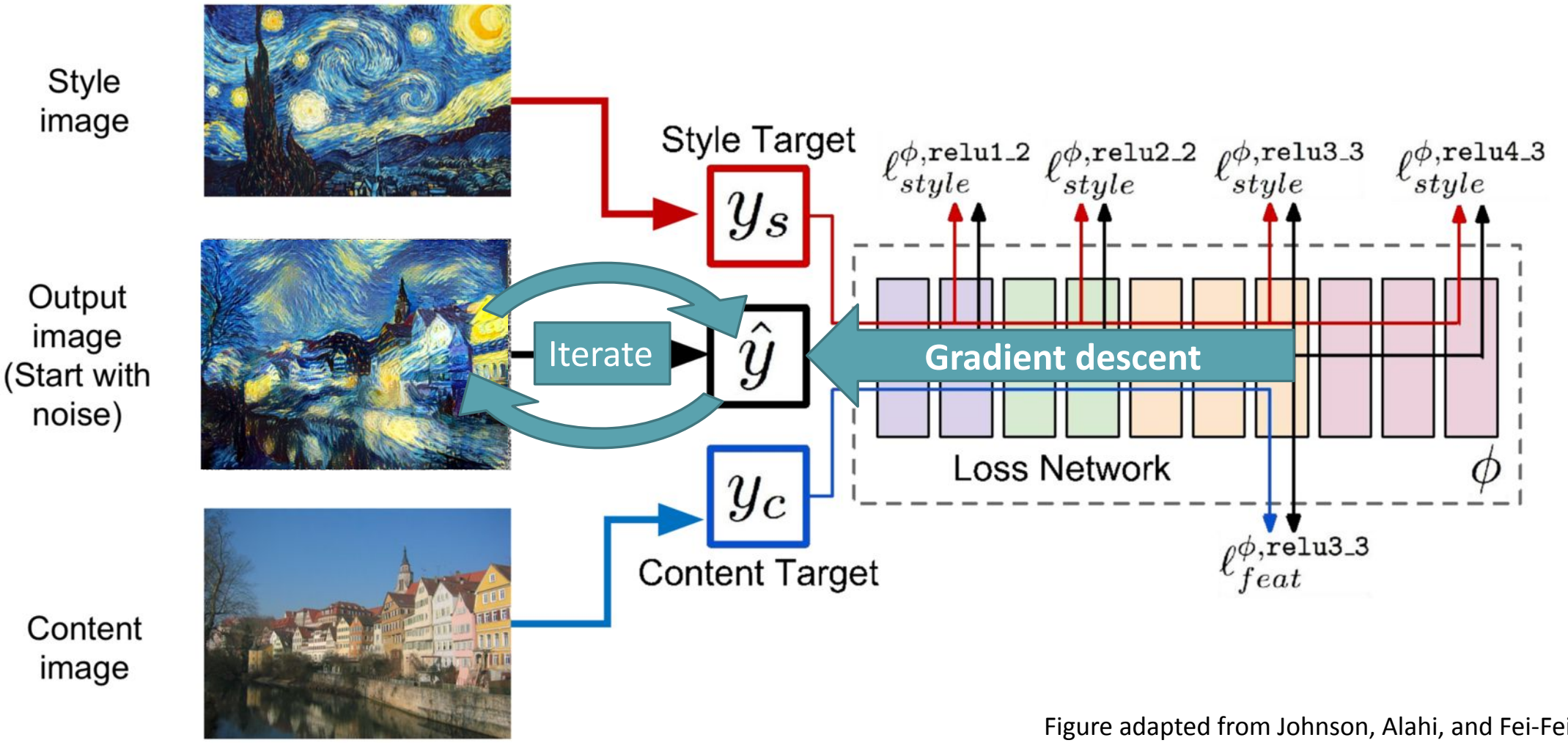


Figure adapted from Johnson, Alahi, and Fei-Fei, *ECCV* 2016

Iteration makes it slow in inference!

Recap: style transfer

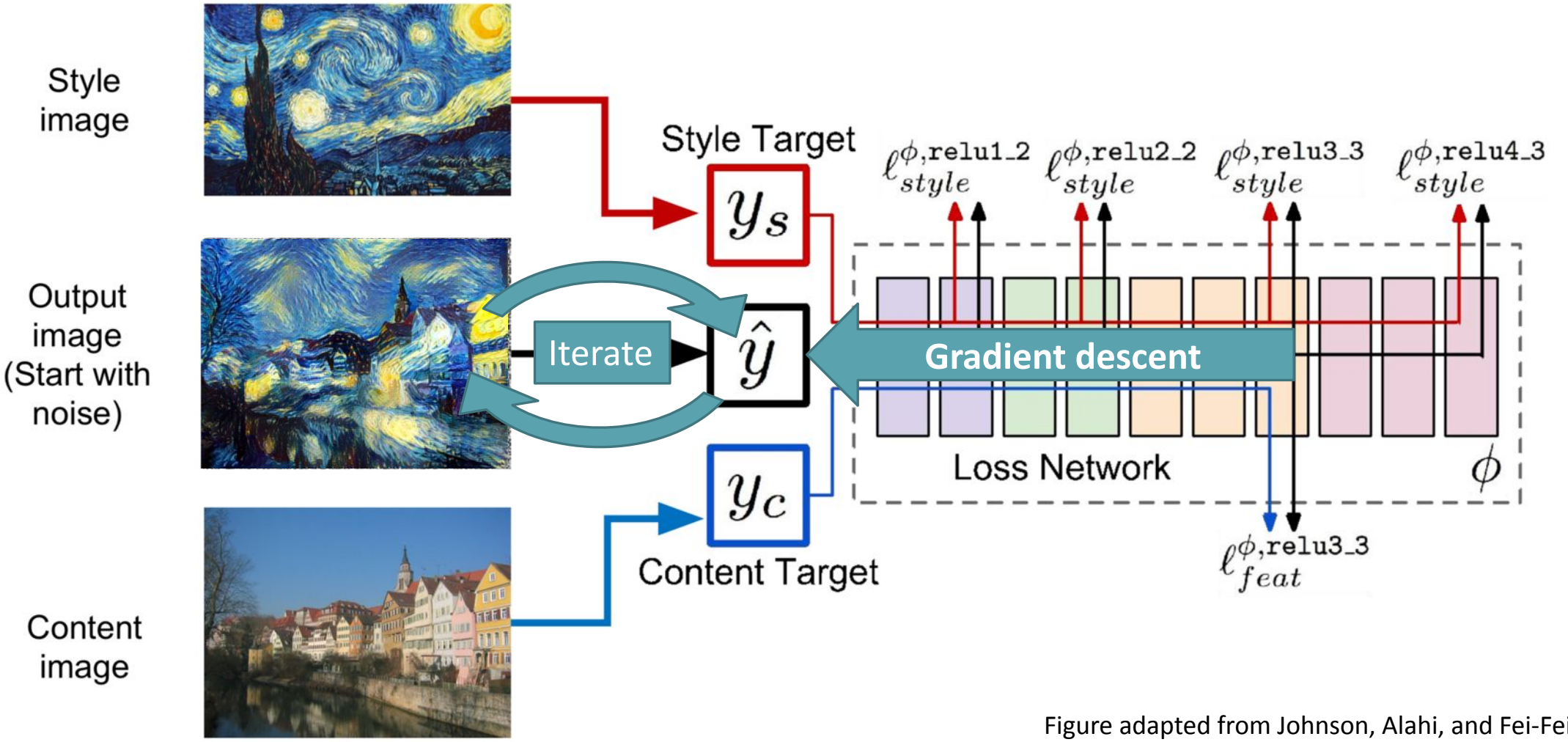
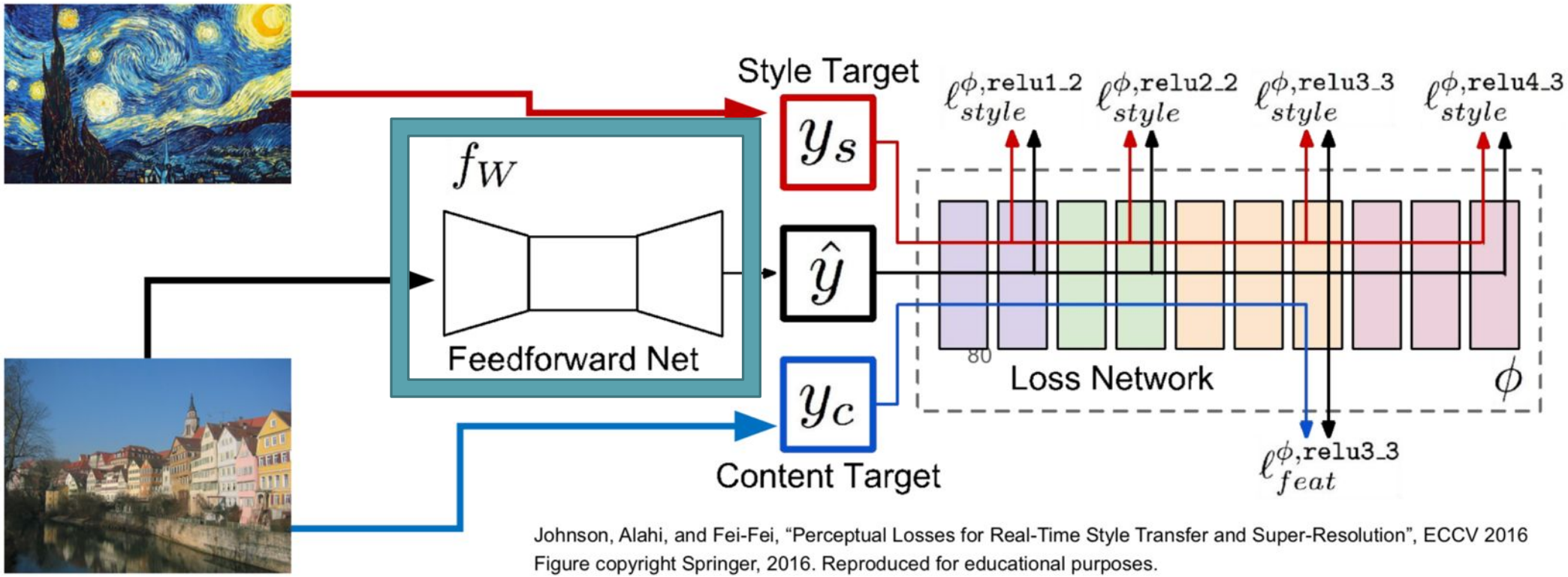


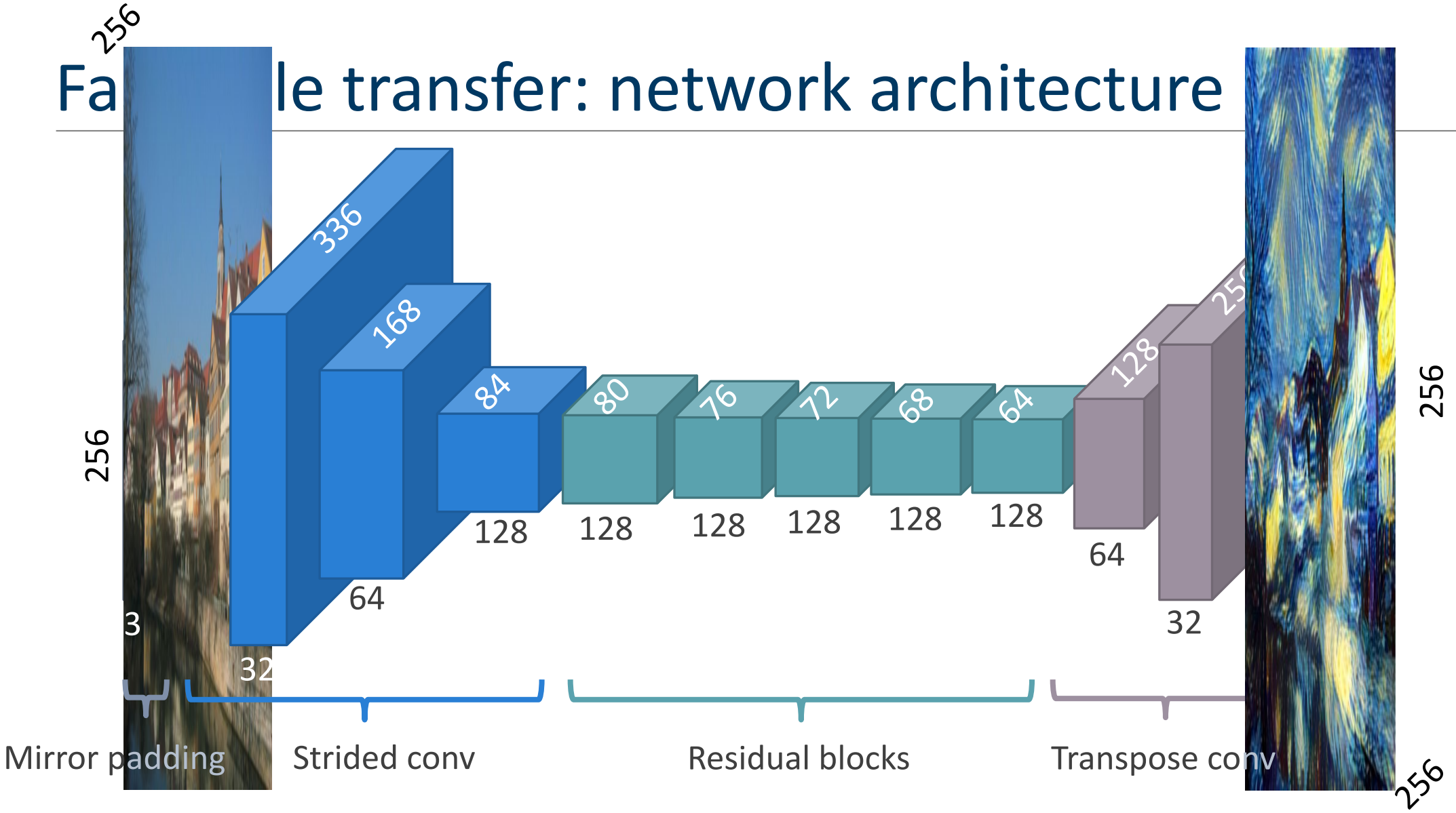
Figure adapted from Johnson, Alahi, and Fei-Fei, *ECCV* 2016

Fast style transfer



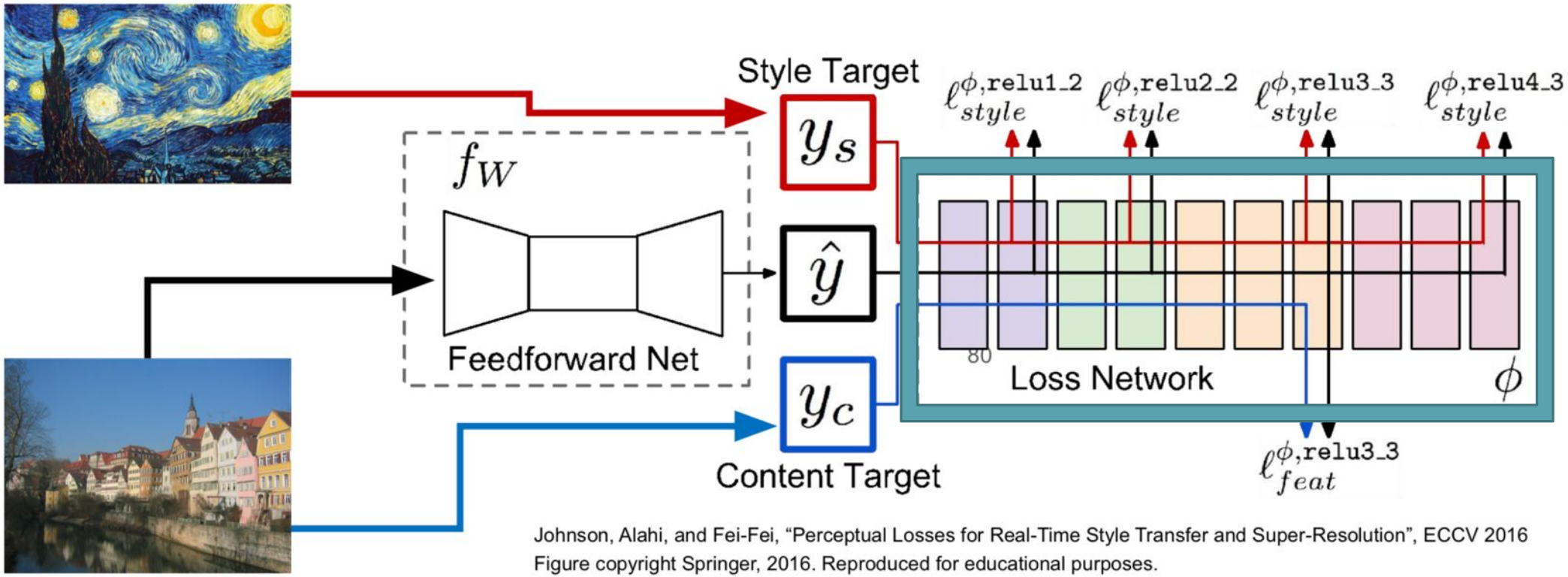
Johnson, Alahi, and Fei-Fei, "Perceptual Losses for Real-Time Style Transfer and Super-Resolution", ECCV 2016

Fa le transfer: network architecture



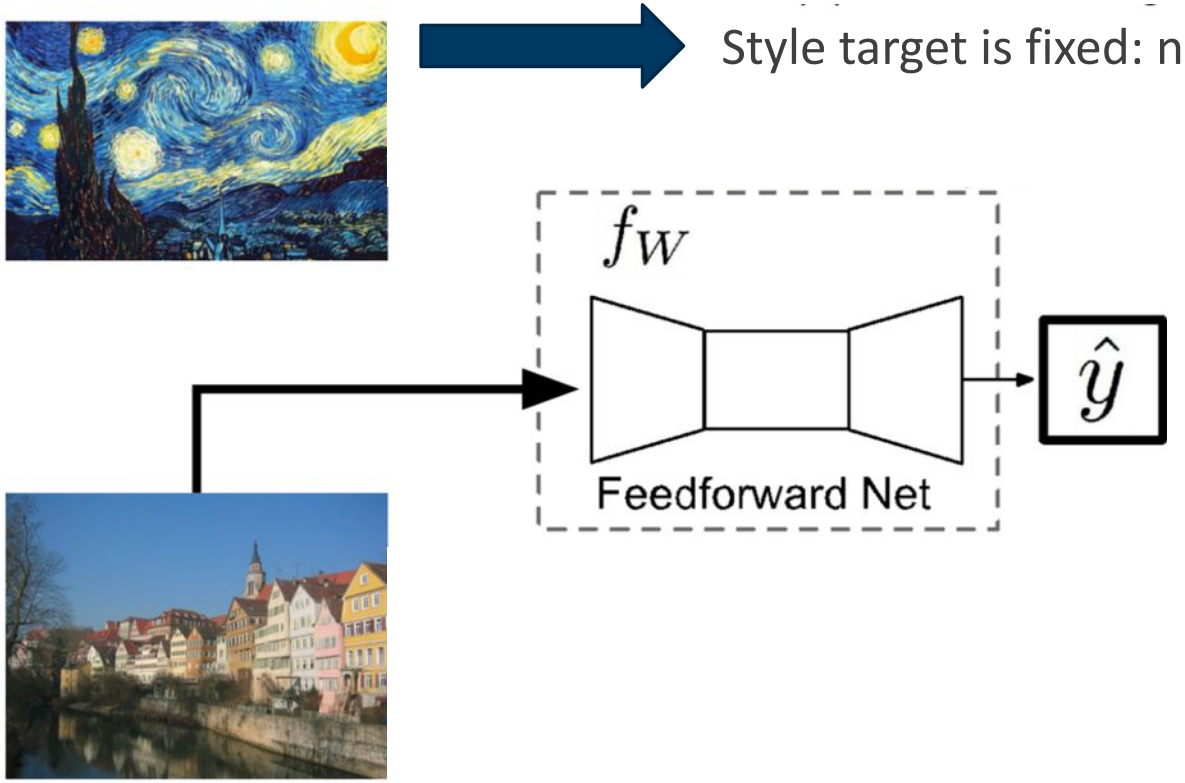
Johnson, Alahi, and Fei-Fei, *ECCV* 2016

Fast style transfer: training



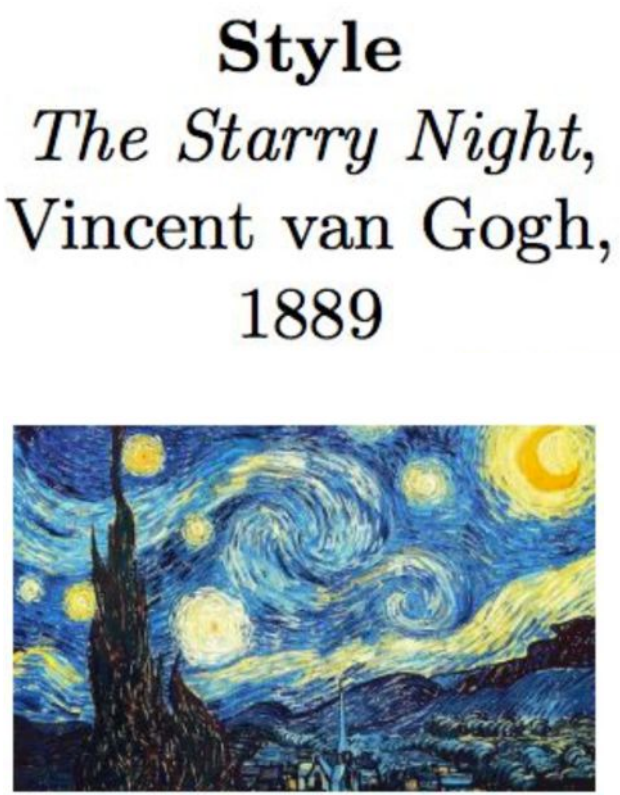
Johnson, Alahi, and Fei-Fei, "Perceptual Losses for Real-Time Style Transfer and Super-Resolution", ECCV 2016
Figure copyright Springer, 2016. Reproduced for educational purposes.

Fast style transfer: image generation



Johnson, Alahi, and Fei-Fei, *ECCV* 2016

Fast style transfer: results



Iterative
(Gatys, Ecker, Bethge 2016)

Fast style transfer

Johnson, Alahi, and Fei-Fei, *ECCV* 2016

Fast style transfer: results

Style
The Muse,
Pablo Picasso,
1935

Iterative
(Gatys, Ecker, Bethge 2016)

Fast style transfer

Johnson, Alahi, and Fei-Fei, *ECCV* 2016

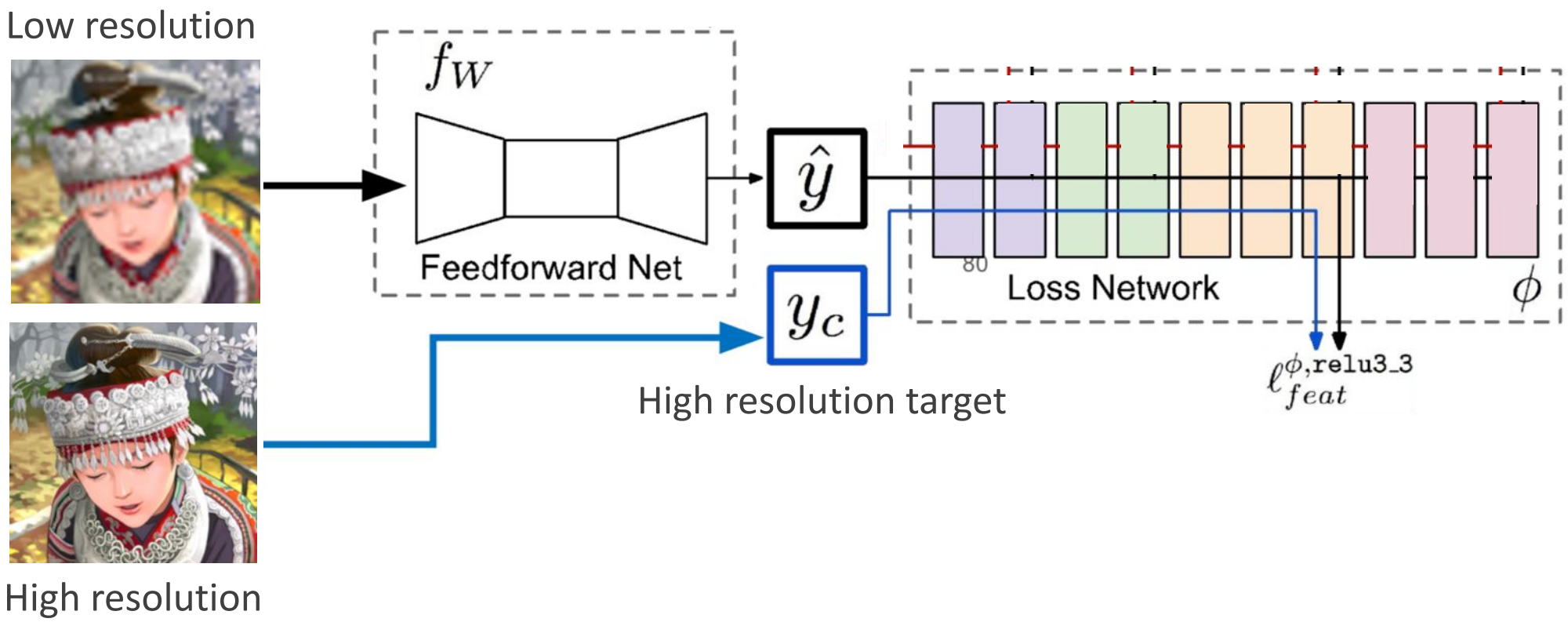
Neural style transfer implementations

Leon Gatys' PyTorch implementation:
https://github.com/leongatys/PytorchNeuralStyleTransfer

Justin Johnson's excellent and very popular Lua/Torch implementation:
(lots of features, but may be hard to run these days)
https://github.com/jcjohnson/neural-style

Neural Style in official PyTorch tutorials:
https://pytorch.org/tutorials/advanced/neural_style_tutorial.html

Super resolution: same net + content loss

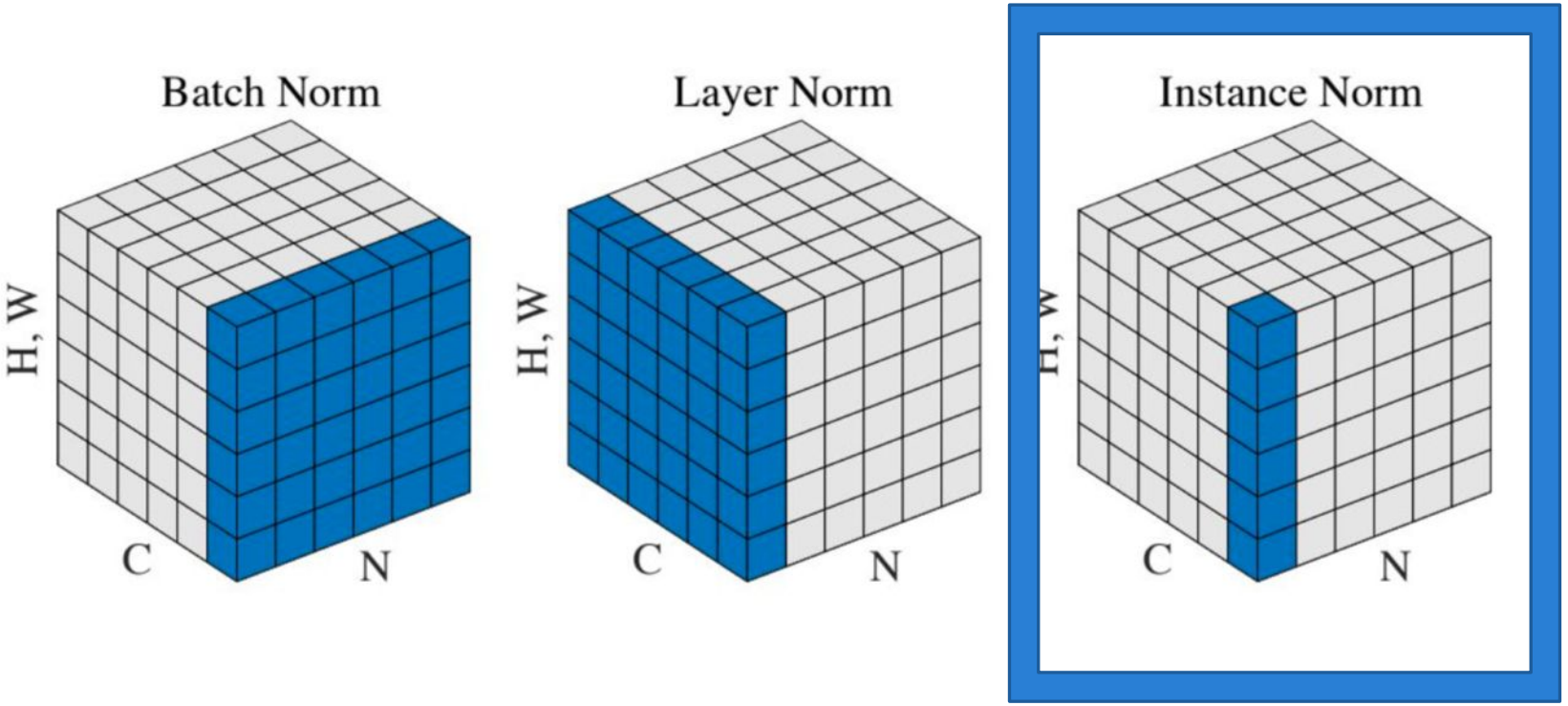


Johnson, Alahi, and Fei-Fei, *ECCV* 2016

Super resolution: Perceptual loss delivers strong performance

Johnson, Alahi, and Fei-Fei, *ECCV* 2016

Remember instance normalization? It was invented for fast style transfer



Ulyanov, Vedaldi, Lempitsky, arXiv 2016

Slide credit: Fei Fei Li, Justin Johnson, Serena Yeung

Fast style transfer with instance norm

Same architecture as before

Replacing batch norm by Instance norm improves stylization results

Ulyanov, Vedaldi, Lempitsky, arXiv 2016

Fast style transfer with instance norm

Photograph

Style

Johnson et al. 2016

With instance norm

Ulyanov, Vedaldi, Lempitsky, arXiv 2016

Arbitrary fast style transfer

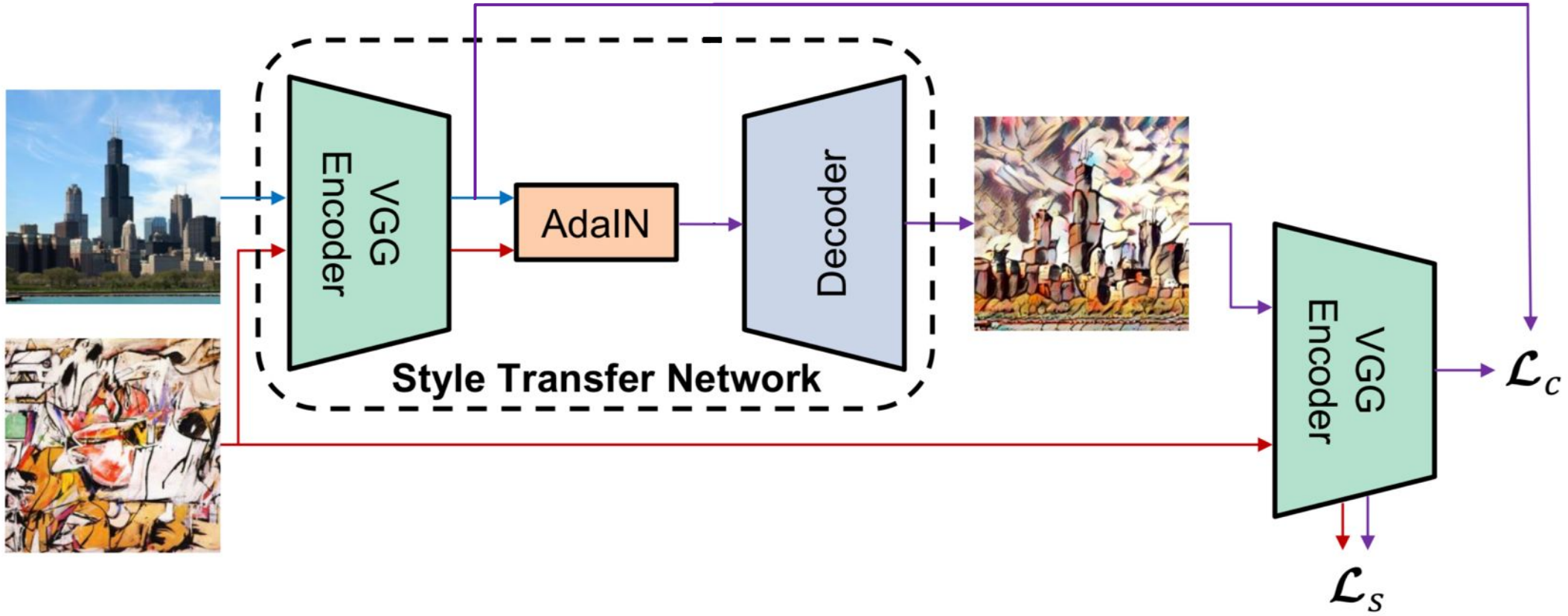
- Neural Style Transfer:
  - style transfer is slow (iterative gradient descent)
  - can transfer arbitrary style

- Fast Neural Style Transfer:
  - style transfer is fast (single pass)
  - need to train separate network for each style

- Can we achieve both?
  - fast inference for arbitrary style

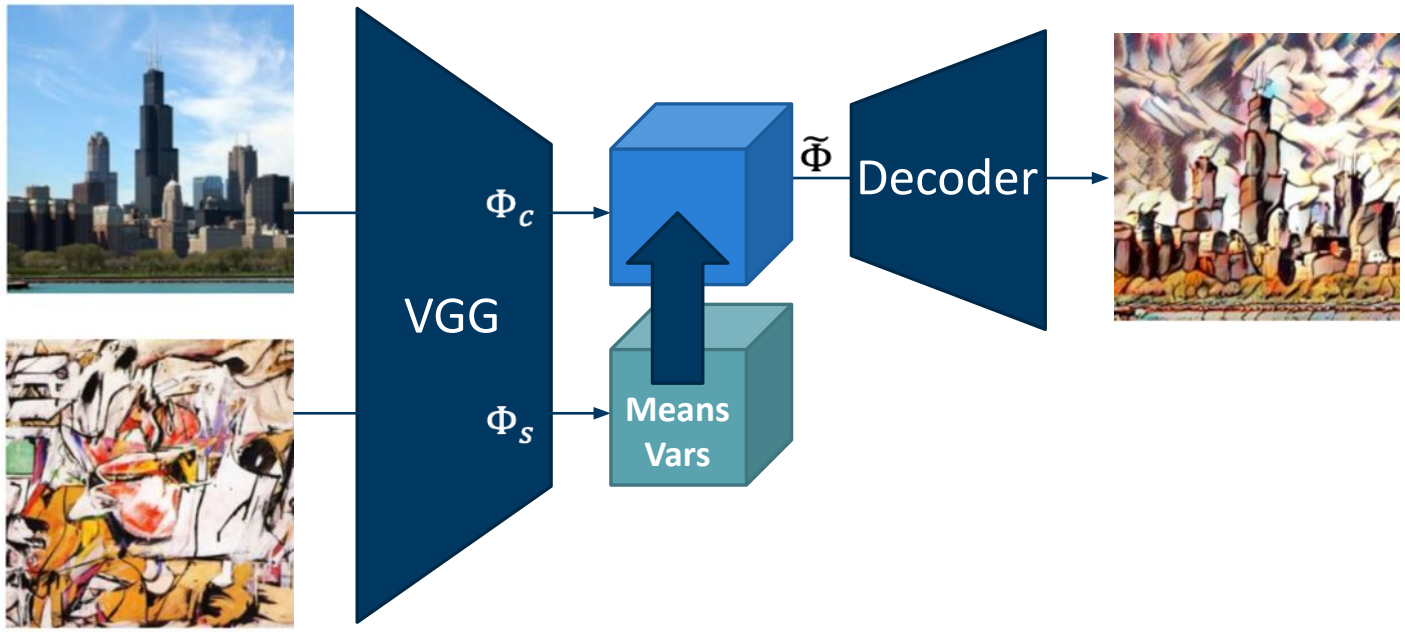
Huang & Belongie, *ICCV* 2017

Arbitrary fast style transfer: Adaptive instance normalization (AdaIN)



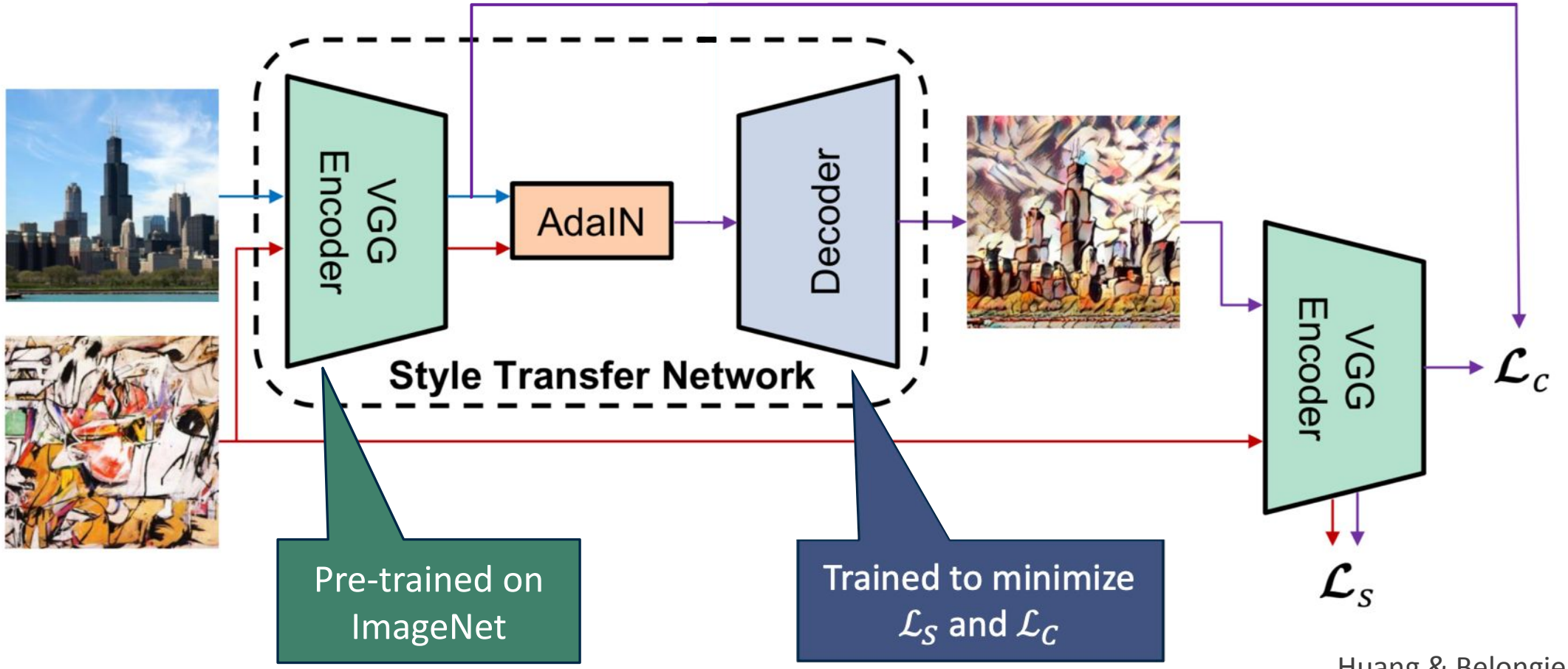
Huang & Belongie, *ICCV* 2017

AdaIN



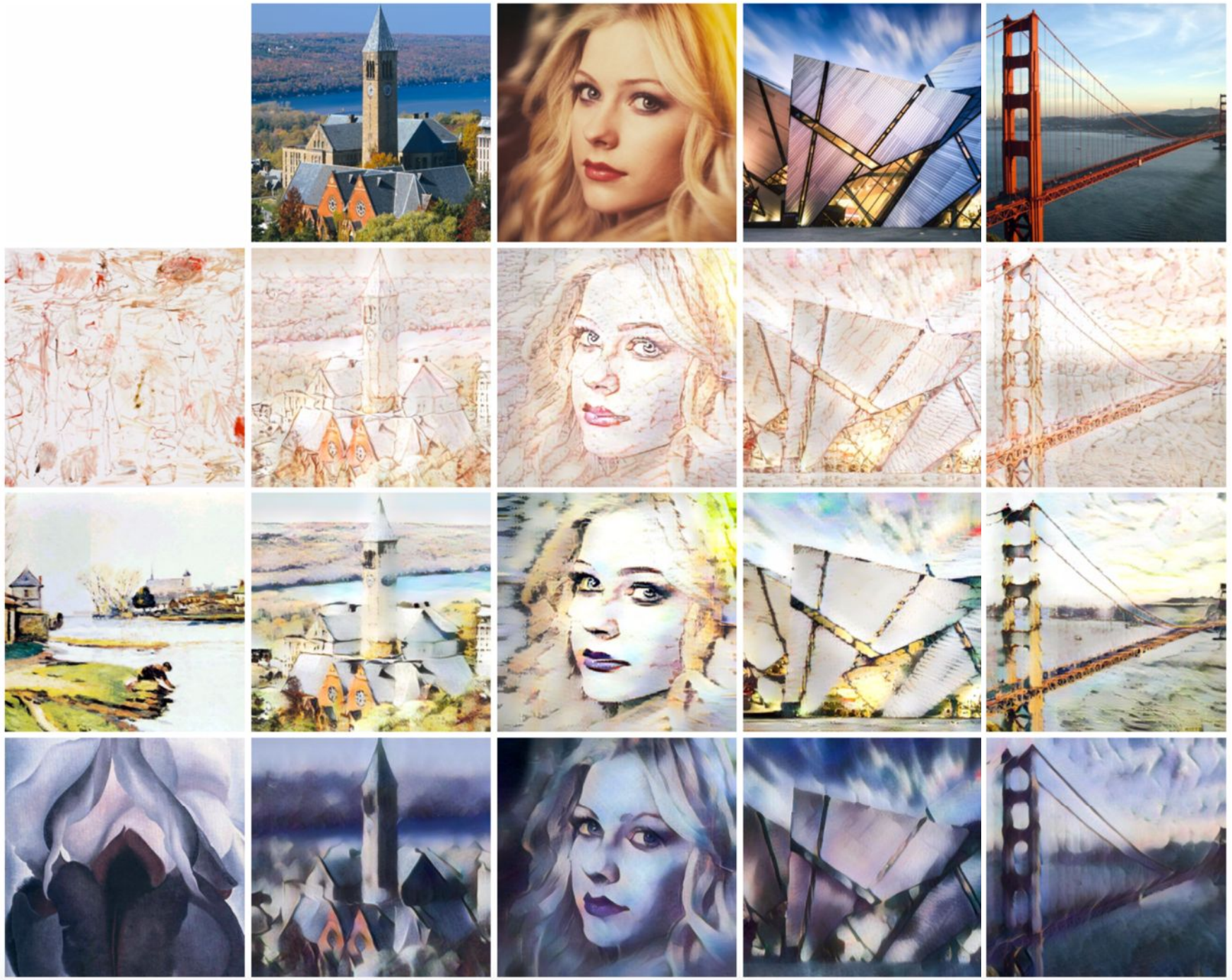
$$\tilde{\Phi}(:,:,\mathrm{i}) = \frac{\Phi_c(:,:,i) - \mu_c(i)}{\sigma_c(i)}\sigma_s(i) + \mu_s(i)$$

Arbitrary fast style transfer: Adaptive instance normalization (AdaIN)



Huang & Belongie, *ICCV* 2017

AdaIN results



Huang & Belongie, *ICCV* 2017

Questions!